Boolean Index Alignment: Traceable Induction on Scales for Sparse Language-Model Systems

Author: Manuel Coleman[1]
[1]AgreeYa Solutions (Computacenter), Folsom, CA, USA
Correspondence: manuel.coleman@agreeya.net
Date: 2026-11-05 (America/Los_Angeles)

Prior-work anchors. This is a continuation of Coleman (2026a), Index Alignment in Large Language Models: Sparse Routing, State Ledgers, and Generative Continuity, and a carefully bounded systems use of the scale analogy in Coleman (2026b), Kakeya Grain at the Slab: Induction on Scales, Continuous Direction, and Prime-Carrying Custody under Love-Squared Coherence.

Abstract

Sparse language-model systems make sequences of discrete choices: expert identifiers, agent identifiers, retrieval locations, tool names, memory paths, and provenance links. Coleman (2026a) called their mutual consistency **index alignment** and separated non-authoritative routers, state ledgers, and generative continuity experts. This paper turns that framework into a finite, inspectable decision construction. At every declared scale we ask four evidence-conditioned questions: are the activated indices live, permitted, and task-admissible (`A_route`); are the associated commitments authorized, provenance-complete, correctly ordered, and non-silently superseded (`A_ledger`); do externally dependent generated claims resolve through activated indices and supported claim paths (`A_gen`); and does the continuation preserve the accepted ledger view, open obligations, and correction scope (`A_cont`)? Their conjunction is `A(k)`, with an explicit PASS-only release bit and retained UNKNOWN or NO_VERDICT diagnostics when observations or caller contracts are absent. A finite trace is the vector of these outcomes, and `FIRST_FAIL` is the least scale at which a required conjunct fails. Under a finite total scale order, finite enumeration, deterministic non-vacuous checking, and a declared sub-conjunct order, the first failed scale and reported failing conjunct are unique and computable; on causal DAGs, concurrent failures are instead returned as a frontier set. This is a property of the construction, not a theorem imported from harmonic analysis.

The geometric analogy from Coleman (2026b) supplies an organizing discipline rather than an implication: begin with a sticky or single-parent case, isolate a non-sticky band, factor only that band, recurse with a declared cap, and account for the residual. Wang–Zahl and Guth–Wang–Zahl use asymptotic inequalities with losses; they do **not** prove that alignment is boolean. Booleanity here is imposed by caller-supplied thresholds and explicit checkers. We develop typed index lineages, scale trees, correction-invalidation

semantics, sticky and non-sticky cases, bounded-refactoring recursion, localization and composition lemmas, and the proposed decision object `INDEX-ALIGN-CERT`, whose outcomes are `{CERTIFIED, HELD_PARTIAL, REFUSED, AWAITING_HUMAN, NO_VERDICT}`. Restricted certificate verification is polynomial in the supplied trace, chunks, claims, and edges; this does not classify the search problem or assert NP-hardness. Worked traces cover MoE routing, retrieval-grounded generation, tool and agent handoffs, stale evidence, revoked authority, correction propagation, false coverage, and multiscale partial release. The protocol separates mathematical background, formal construction, synthetic tests, registered observations, and unperformed live evaluation. The fourteen traces and thirty-two mutations are designs only, not test executions. Coverage is not fidelity, order is not authority, custody is not truth, and `Act = 0` does not entail `Stop`.

Keywords: index alignment; boolean certificates; traceability; induction on scales; sparse routing; mixture of experts; mixture of agents; retrieval-augmented generation; state ledgers; provenance; correction propagation; failure localization; decision procedures

---

1. Introduction

1.1 From a consistency framework to an executable question

A sparse system can be fluent, preference-aligned, and internally confident while selecting the wrong expert, citing a document it never retrieved, executing a stale tool result, or carrying a corrected fact into an obsolete continuation. Coleman (2026a) organized these failures around discrete **indices** rather than treating every failure as an undifferentiated generation defect. Its central object was the consistency of a routed set `R_t`, a ledger prefix `L_t`, generated text `y_{1:t}`, and continuity state `C_t`. Its architecture separated a router that proposes activation, a ledger that accepts durable commitments, and a continuity specialist that preserves plans and discourse without acquiring factual authority.

That separation makes a stronger question possible. Instead of asking only for a scalar alignment deficit, we can ask, at a declared scale, **which exact consistency condition holds?** A scale can be a token-routing layer, a retrieval-to-claim stage, a tool-call boundary, an agent handoff, a session resume, or a document-chunk refinement. The proposed response is not a single opacity-reducing score. It is a finite vector of booleans, each tied to a witness and each capable of failing independently. The trace can therefore answer both “is this trace certified under the declared contract?” and “where, in which relation, did certification first become impossible?”

This is the precise sense in which **alignment can be treated as a boolean kind of problem**. The adjective does not mean that semantic correctness is effortlessly binary, that a neural hidden state has a canonical truth value, or that all model behavior can be decided by logging identifiers. It means that a designer can declare finite, checkable sufficient conditions for a restricted notion of index consistency, supply their thresholds

and evidence, and return an auditable yes, no, or no-verdict. A semantic proposition whose support cannot be checked remains unresolved; the framework forbids laundering that uncertainty into `True`.

1.2 Why induction on scales is useful as an analogy

Coleman (2026b) emphasizes the reading order sticky, then non-sticky control by induction on scales, then the general theorem-background in three-dimensional Kakeya geometry. In that external literature, slabs, planks, grains, and other carriers factor incidence structure while preserving ancestry across refinements. The full-dimensional Kakeya result in $\mathbb{R}^3$ is theorem-background associated with Wang–Zahl (2025) and Guth–Wang–Zahl (2026). Those authors are not affiliated with this work and their mathematics proves nothing about language-model systems.

The systems analogy has three disciplined parts. First, begin with a **sticky** case in which every active fine item has one visible coarse parent, provenance is shared, and local checks are comparatively simple. Second, identify a **non-sticky** band where ancestry, authority, support, or correction propagation differs across children; factor only that band rather than restarting the entire trace. Third, make the general result conditional on termination, explicit round caps, and residual accounting. The analogy changes how we organize an audit; it does not supply the audit's semantic predicates or prove their validity.

There is also an important disanalogy. External Kakeya arguments use asymptotic inequalities such as $\lesssim$, with losses and source-specific normalizations. A certificate, by contrast, must eventually compare a measured or declared quantity with a caller-selected threshold. Thus `STICKY_k := KT_k ≤ θ_KT ∧ F_k ≤ θ_F` is a computational definition, not a quotation of a geometric theorem. No threshold yields `NO_VERDICT`, never a default pass. Paper grains are not SAVER grains; a geometric scale ratio is not an authority score; and neither “continuous direction” nor “prime velocity” is needed as a mathematical primitive here.

1.3 Contributions

This paper contributes the following.

1. **Typed boolean alignment.** We define routing, ledger, generation, and continuity predicates over a common typed trace, together with `A(k)`, per-scale witnesses, and five-valued certificate outcomes.
2. **A finite scale-induction schema.** Sticky base case, non-sticky band factoring, recursion, and residual handling are defined without claiming a Kakeya-to-LLM implication.
3. **Elementary localization results.** We prove uniqueness of the first failing scale, finite computability, non-vacuity requirements, refinement and correction-invalidation properties, and bounds for the restricted verifier.
4. **Architecture mapping.** Non-authoritative routers, state ledgers, and generative continuity experts from Coleman (2026a) become separately testable conjuncts rather than an unscored role taxonomy.

5. **A correction semantics.** Authorized corrections invalidate dependent certificates by graph reachability, preserve history, and require revalidation at affected scales. Ordering alone never grants authority.
6. **Decision and complexity boundary.** `INDEX-ALIGN-CERT` has an explicit input, witness, outcome set, and verifier. Polynomial verification of a supplied restricted witness is distinguished from the unclassified task of finding a good routing or correcting a model.
7. **Worked boolean traces and evaluation protocol.** Fourteen traces expose successes, first failures, concurrent frontiers, partial release, and no-verdict conditions. A future benchmark design specifies mutations, oracle labels, inter-rater requirements, cost accounting, and falsifiers. No new empirical result is reported.

1.4 What is actually established

The paper proves finite-list and graph facts about its own definitions. It does not prove that real LLMs satisfy them, that an LLM's hidden computations are captured by exposed indices, or that passing all checks makes an answer true. “Index alignment” remains distinct from preference alignment. A certificate says that a declared relation among recorded indices, evidence, commitments, and continuation passed a declared checker. It does not say that the underlying world model is correct.

The distinction is visible in two separations carried throughout. **Coverage is not fidelity:** every source chunk may have a valid content-bound receipt while the conclusion depends on an unsupported claim. **Order is not authority:** an event log can show that one write happened before another without showing that either writer was authorized. A third separation, inherited from Coleman (2026b), is that custody is not truth: an object can be retained for evaluation while its central hypothesis remains open.

1.5 Paper organization

Section 2 situates the work relative to both former papers and external sparse-computation literature. Section 3 defines indices, events, ledgers, scale trees, claim graphs, and evidence. Section 4 defines the four predicate families and composite alignment. Section 5 gives the induction schema. Section 6 proves localization and related propositions. Section 7 maps the construction to the role architecture. Section 8 defines correction invalidation and release outcomes. Section 9 presents worked traces. Section 10 specifies the decision object and verification bounds. Section 11 gives the evaluation protocol and falsifiers. Section 12 discusses failure modes and deeper research questions. Section 13 states limitations and non-claims, and Section 14 concludes. Appendices supply certificate schemas, pseudocode, notation, mutation inventories, claim-status registers, and a source-to-construction map.

---

2. Relation to the Two Former Papers and External Literature

2.1 Coleman (2026a): what is retained and what changes

Coleman (2026a) defines four requirements: routing–capability consistency, routing–ledger consistency, generation–index consistency, and continuity–ledger consistency. It models a sparse routing decision as `R_t = {(ι_j,g_j)}`, a ledger as append-oriented records with provenance and validity, and a continuity state containing entities, constraints, open goals, and discourse commitments. It then introduces an alignment deficit `Δ_t` as a weighted sum of four distances. That scalar is useful for optimization and aggregate reporting but insufficient for diagnosis: a weighted average can hide which required relation failed and can permit one strong component to compensate for another.

This paper preserves the four-way decomposition but replaces the compensatory release rule with a non-compensatory boolean contract. `A_route`, `A_ledger`, `A_gen`, and `A_cont` correspond respectively to those four requirements. The scalar deficit may still be reported as telemetry, but no positive amount of routing quality can compensate for missing provenance, and no citation coverage can compensate for an unauthorized commit. A failed required conjunct prevents `CERTIFIED`; depending on scope it can still permit `HELD_PARTIAL` that names the failure.

The earlier role taxonomy also becomes operational. A router's non-authoritativeness is tested by the absence of direct ledger commit capability and by the provenance of subsequent proposals. A ledger's authority is tested by transition admissibility, not by the fact that it is called a ledger. A continuity expert's projection is tested against the committed view and correction scope rather than inferred from fluent prose. Capability isolation is therefore not a visual architectural feature; it is an obligation in the boolean predicates.

Finally, the illustrative IA-MoA algorithm is refined into a checker that can run alongside an existing system without requiring a new foundation model. The immediate research object is not a claim of better generation. It is whether real traces can be labeled consistently, whether first-failure localization predicts the repair site, and whether partial release reduces unsupported promotion without excessive false refusal.

2.2 Coleman (2026b): what the analogy licenses

The Kakeya-grain white paper distinguishes theorem-background, formal typed objects, proposed constructions, and measured observations. Its scale order is sticky/common-center observability, non-sticky sparse multiscale control, slab/plank factoring, residual accounting, and only then the general result. The present paper imports that order as a structural analogy for audit organization. It does not import a mathematical conclusion.

The analogy suggests four engineering rules. Preserve ancestry when a fine index is regrouped under a coarse parent. Factor a failing band rather than averaging its failure into global telemetry. Record residual items that no admissible carrier explains. Never

reverse the inference order by starting with the desired aligned conclusion and treating it as a property of the underlying geometry. These are methodological choices for systems use.

The Kakeyalogic round-two seat packet makes the boundary exact: boolean/traceable induction on scales is not a consequence of Wang–Zahl or Guth–Wang–Zahl. The mathematical sources prove asymptotic volume bounds with losses; per-scale inequalities are not exact booleans. Thresholds belong to the data owner or evaluation designer. A sticky KNS fan, moreover, is a typed observability object and is not identical to the sticky case in the external Kakeya proof. Our computational sticky case is a third, separately defined object.

Other terms in Coleman (2026b) stay outside the proof core. `HELD` motivates retained partial output, not factual endorsement. `Act = 0 ⇏ Stop` motivates the distinction between blocking promotion and suppressing all output. `h < 1` remains a documented evaluator-non-sovereignty boundary, not a calibrated score. L^2_C authority detection informs the rule that source, authority, intent, and harm must be distinguished before incorporation. SAVER supplies a neighboring non-compensatory design discipline, but paper grains and SAVER grains are different objects. Prime-carrying and Riemann burdens are not translated into routing variables.

2.3 Sparse experts and routers

Sparsely gated MoE introduced conditional expert activation at controlled compute (Shazeer et al., 2017). GShard used top-2 expert routing and automatic sharding (Lepikhin et al., 2020); Switch Transformers simplified the mechanism to top-1 (Fedus et al., 2022); Mixtral used top-2 among eight experts in MoE layers (Jiang et al., 2024). Surveys and refactorization work explore alternative gates, dynamic budgets, and router decoupling (Cai et al., 2024; Yue et al., 2024; Read-ME, 2024). These systems supply observable expert IDs, gates, and sometimes auxiliary losses.

Index alignment does not declare an oracle expert from routing entropy alone. A low-entropy or collapsed route can be valid for a narrow task; a balanced route can be wrong. `A_route` therefore separates registry validity, permission, budget, and declared task admissibility. Learned-router quality remains an empirical question. The certificate's role is to expose which of those checks failed, not to claim that an expert's output is semantically correct because its identifier was permitted.

2.4 Pointers, retrieval, and attribution

Pointer Networks make outputs indices into an input set (Vinyals et al., 2015). RAG conditions generation on retrieved evidence (Lewis et al., 2020), while retriever ranking, evolving graph indices, and self-learned indexers further expose retrieval choices (LTRR, 2025; GraphAnchor, 2026; PG-RAG, 2024). Our generation predicate is a multi-store extension of pointer discipline: a cited index must have a typed lineage back to activated evidence, and the claim depending on it must have adequate support. Restricting citations to active IDs prevents one failure—inventing an inactive identifier—but does not prevent fabricating content under a real identifier. `A_gen` therefore requires both lineage and a support status; support may itself be `UNKNOWN`.

2.5 Tools, agents, and durable state

Toolformer and ReAct expose API calls and action–observation steps (Schick et al., 2023; Yao et al., 2023). AutoGen, MetaGPT, and Magentic-One provide multi-agent orchestration, with the latter using structured ledgers and specialized workers (Wu et al., 2023; Hong et al., 2023; Fourney et al., 2024). MemGPT, Generative Agents, transactional agent memory, and path-addressed stores expose different memory and state disciplines (Packer et al., 2023; Park et al., 2023; TARL, 2026; StateLedger, 2026). The present construction is compatible with these systems but does not assume that their logs already satisfy its contract. In particular, timestamps and Lamport clocks can support `ORDER`, while external evidence is required for `AUTH`.

2.6 External geometry and the object firewall

Wang–Zahl (2025) and Guth–Wang–Zahl (2026) establish full Minkowski and Hausdorff dimension for Kakeya sets in $\mathbb{R}^3$, here used only as theorem-background for the analogy. Slabs, planks, grains, density, and multiplicity remain source-scoped mathematical terms. Wang–Wu (2024) supplies restriction-estimate and two-ends background. None supplies an alignment predicate, a correction rule, or a result about expert selection. We use no geometric formula as evidence for `A(k)`. The formal work below begins again from finite sets, traces, graphs, and caller-specified predicates.

3. Formal Objects

Definitions 1–17 are computational objects introduced here, apart from ordinary finite-set and graph terminology; none is attributed to external geometry. All thresholds are caller-supplied, and absent mandatory inputs produce `NO_VERDICT`.

3.1 Typed index spaces and registry epochs

Definition 1 (Typed index). Let `□_type = {expert, agent, tool, doc, memory, claim, correction, source, obligation}` be resource types. A typed index is

`ι = (type(ι), name(ι), epoch(ι), version(ι)),`

drawn from a finite disjoint union `□ = ⋃_τ □_τ`. The epoch distinguishes a registry state; the version distinguishes the resource contents or interface. An index name without a type is not a well-formed certificate field. Two resources with the same display name but different types or versions are different indices.

A registry entry is

`Reg(ι) = (type, capabilities, input_schema, output_schema, authority_scope, validity_interval, digest).`

`Live_k(ι)` means that the entry exists at scale `k`, its validity interval covers the relevant logical time, and its digest matches the observed artifact. `Permitted_k(ι,a)` means that capability policy `Π_k` permits action `a` on that entry for the authenticated principal. The policy is an input to the audit, not something inferred from successful execution.

Definition 2 (Activation). An activation record is

```
r = (ι, parent, gate, input_digest, output_digest, principal, event_id),
```

where `parent` is a typed parent index or `⊥`, and `gate` is optional. The active set at event `t` is `R_t`. `R_{≤t}` is the set of activations visible to event `t` under a declared snapshot rule. A gate weight is telemetry; a nonzero gate does not imply permission, authority, support, or correctness.

Definition 3 (Index morphism and lineage). A typed morphism `μ : □_a → □_b ∪ {⊥}` records a legal cast, such as document to memory path or tool output to ledger subject. A lineage for index `c` is a path

```
ι_0 →^{μ_1} ι_1 →^{μ_2} … →^{μ_n} c
```

with `ι_0 ∈ R_{≤t}` and every edge accompanied by an event witness. `⊥` denotes a failed cast. `Lin_t(c)` is the set of such witnessed paths. If no path exists, `c` is unaligned with the activation history even when its display name is real in some registry.

3.2 Sources, chunks, and content-bound receipts

Definition 4 (Source units and version). A source `S` is a finite sequence `u_1,…,u_N`. Each unit has a content digest, source version, intake event, and source authority record. SHA-256 can bind bytes to an identifier; it does not show that a model read, understood, or was entitled to use the bytes. A **content-bound receipt** is

```
R(u) = (digest(u), version(u), quoted_span, span_offset, verifier_id, intake_event, vector_clock, scope).
```

The verifier-chosen quoted span binds the receipt to content rather than merely to file presence. Independent verification requires a verifier outside the process capable of forging the receipt. In-process receipts are construction evidence only.

Definition 5 (Scale tree). A scale ladder is `□ = (s_0 > s_1 > … > s_M)`, with finite `M`. Each `P_k` is a partition of the source units or indexed working set into chunks, and `P_{k+1}` refines `P_k`: every `C ∈ P_{k+1}` has a unique `parent(C) ∈ P_k` with `C ⊆ parent(C)`. A node may additionally point to routing, ledger, generation, and continuity events in its scope. `children(C)` and `Desc(C)` have their usual finite-tree meanings. The root represents ambient context; leaves represent the finest declared units, not unknowable token-level neural semantics.

No formula for `M` is imported from geometry. The implementation declares a finite depth appropriate to its source inventory and audit budget. Uniform branching means that within-level child counts agree up to a declared factor; it is not a precondition for the finite localization results.

Definition 6 (Density observables). For a selected set `Q_k` of level-`k` chunks and an admissible family of coarse windows `□_k`, let `Expected_k(W)` be the data owner's expected selected count under a declared allocation model. Define

```
KT_k = max_{W∈□_k} |{C∈Q_k : C⊆W}| / Expected_k(W),
```

only where the denominator is positive and the window family is specified. For each chunk `C`, let `density_k(V)` be supported selected mass in sub-span `V` under a declared measure. Define

F_k = max_{C∈P_k} max_{V⊆C, density_k(C)>0} density_k(V)/density_k(C).

These are **computational analogues** of clustering and concentration observables. They are not the Katz–Tao or Frostman quantities themselves. When denominators, measures, or expected counts are unspecified, KT_k and F_k are UNKNOWN.

Definition 7 (Boolean stickiness). Given thresholds θ_KT, θ_F,

STICKY_k(θ) := [KT_k ≤ θ_KT] ∧ [F_k ≤ θ_F].

STICKY(θ) := ∀k, STICKY_k(θ). A single-parent special case may additionally require every fine activation to inherit the same registry epoch, authority scope, and correction scope as its coarse parent. Stickiness is a routing and custody condition; it neither grants action authority nor identifies the sticky case of an external Kakeya proof.

3.3 Ledger and authority objects

Definition 8 (Ledger record). A ledger L is a finite append-oriented sequence of records

ℓ_j = (τ_j, a_j, subject_j, value_j, provenance_j, status_j, authority_j, supersedes_j, scope_j, event_id),

where a_j ∈ {ADD, PEND, REJECT, REVISE, DEFER, SUPERSEDE}. provenance_j is a nonempty set of typed source lineages for an accepted record. authority_j contains authenticated issuer evidence and permitted scope. supersedes_j links to earlier records without deleting them. scope_j names affected claims, routes, tasks, and validity interval.

View(L_t) is the committed, non-superseded view at event t under a declared snapshot rule. Pending(L_t) and Rejected(L_t) remain queryable. A ledger transition δ(L,p) consumes a proposal p; it is not made valid merely by being deterministic. A transition is admissible only if the proposer has proposal permission, the acceptor has commit authority, the value satisfies schema and temporal rules, and every replacement preserves a supersession chain.

Definition 9 (External authority evidence). For action x, let S(x) denote authenticated source authority, H(x) hierarchy validity, and U(x) authorized user intent, each in {0,1,UNKNOWN}. Define

Act(x) = 1 iff S(x)=H(x)=U(x)=1.

If any factor is 0, Act(x)=0; if none is 0 and at least one is UNKNOWN, Act(x)=UNKNOWN. This three-valued elaboration preserves the L-seat product rule while preventing missing evidence from becoming a pass. S(x)=0 ⇒ Act(x)=0; Act(x)=0 ⇏ Stop. Coverage, stickiness, a found receipt, causal order, or model agreement never sets S(x)=1.

3.4 Claims, support, and generation

Definition 10 (Claim graph). A claim graph is a finite directed acyclic graph G=(V,E). Nodes include source units, extracted claims, intermediate inferences, generated claims, obligations, and outputs. An edge u→v means that v depends on u. Every generated externally dependent claim q has a set Cite(q) of typed citation indices and a support label

```
Support(q) ∈ {SUPPORTED, UNSUPPORTED, CONTESTED, UNKNOWN}
```

under a declared verifier. `Deps(q)` includes all ancestors of `q`. The graph can contain unsupported nodes; its acyclicity is a structural condition, not evidence that its contents are true.

A support checker may combine exact extraction, logical rules, human annotation, or an entailment model. Its identity, version, and error profile are part of the evaluation. If it is an LLM, agreement with another LLM is not independent proof. `UNKNOWN` is mandatory when support evidence is out of scope or no applicable verifier exists.

Definition 11 (Generated continuation). A continuation segment `y_k` is represented by `(text_digest, Claims(y_k), Cites(y_k), Obligations(y_k), activated_view, generator_version)`. `Claims` includes only claims within the declared audit schema; ordinary stylistic text need not be converted into factual propositions. `Cites` includes explicit and machine-resolved implicit citations. `activated_view` identifies the snapshot `R_{≤k}` and `View(L_k)` visible to generation.

3.5 Continuity state and obligations

Definition 12 (Continuity state). Let

```
C_k = (C_facts,k, C_disc,k, C_goals,k, C_corr,k),
```

where `C_facts,k` is a projection of `View(L_k)`, `C_disc,k` tracks discourse commitments, `C_goals,k` tracks open and completed goals, and `C_corr,k` tracks active correction scopes. A projection function `π_{C←L}` is declared by the system. Continuity does not require storing the whole ledger in a prompt; it requires that whatever summary is used does not contradict or silently drop in-scope commitments.

An **obligation** is a typed record `(id, condition, required_action, scope, status, source_lineage)`. `status ∈ {OPEN, SATISFIED, WAIVED, UNKNOWN}`; only an authorized record can waive it. Handoffs and resumes generate explicit continuity checkpoints. An open obligation cannot be treated as satisfied merely because later text no longer mentions it.

3.6 Unified event trace

Definition 13 (Indexed event). An event is

```
e_t = (t, scale, R_t, proposals_t, commits_t, y_t, C_t, reads_t, writes_t, actor_t, authority_evidence_t, lineage_edges_t, clock_t).
```

The trace `T = (e_0,…,e_T)` is finite for a certificate run. `scale` maps the event to one or more scale-tree nodes. `clock_t` may include logical and physical timestamps, but `ORDER` uses explicit happened-before edges. A wall clock alone is insufficient across distributed components.

Definition 14 (Happened-before). `HB` is the smallest relation containing explicit program order, message send/receive, intake/receipt, proposal/commit, and correction/dependency edges declared in the trace. Its transitive closure must be acyclic for `ORDER`

to be defined. A cycle in complete relevant evidence yields `ORDER=False`; if completeness or required edges are missing, the result is `UNKNOWN`. `NO_VERDICT` is reserved for an absent ordering contract, not for incomplete observations.

Definition 15 (Correction). A correction is

`c = (target, issuer, authority_evidence, scope, applicability, lineage, nonce, tag, timestamp).`

The tag and nonce support integrity and replay detection in the proposed construction; cryptography does not make the issuer correct. `Authorized(c)` requires authenticated issuer authority over `target` and `scope`. `Dependents(c)` is the reachable set from `target` in `G`, including generated claims, ledger projections, and pending actions marked in scope.

Definition 16 (Scale slice). For scale `k`, `T_k` is the subtrace of events, receipts, claims, and corrections assigned to `P_k`, together with inherited parent context. A slice is **complete for predicate `B`** if every input required by the checker for `B` is present. Otherwise the predicate returns `UNKNOWN`. This prevents a truncated log from being certified as aligned.

Definition 17 (Directional coverage). Let `□_required` be a declared set of distinguishable routes of relevance through state, such as required evidence classes, obligations, or tenant partitions. `Covered_k(d)` means that a distinguishable index lineage for direction `d` remains recoverable at scale `k`. This is the directional-index-continuity analog of the light-rule posture in Coleman (2026b): every required direction through the custody center remains represented. Coverage overlap does not place hidden action and does not imply `AUTH`.

---

4. Boolean Alignment Predicates

4.1 Three-valued checking with boolean certification

Checkers internally return an evidence status `Ev(P)∈{PASS, FAIL, UNKNOWN, NO_VERDICT}`. `PASS` means required evidence is present, fresh, and satisfies the caller contract; `FAIL` means present evidence violates it; `UNKNOWN` means evidence is missing, incomplete, stale, or an assessor is unavailable; `NO_VERDICT` means required thresholds or assessor contracts were not supplied. For readability, the propositions below also write `True`, `False`, and `UNKNOWN` for the first three evidence conditions, while preserving `NO_VERDICT` as a distinct reason code. At the certificate boundary, `b(P)=1` iff `Ev(P)=PASS`, and `b(P)=0` otherwise. This projection does **not** turn unknown evidence into an observed semantic failure. The explanatory trace retains all four statuses. The composite `A(k)=True` only if every required predicate passes; it is `False` if at least one fails; otherwise it is non-pass with the set of `UNKNOWN` and `NO_VERDICT` reasons. Empty optional conjunctions pass only when inapplicability is explicitly established.

4.2 Routing predicate

Definition 18 (`A_route`). For scale `k` and activation `r∈R_k`, define:

1. `Typed(r)`: every index and morphism has a declared type.

2. `Live(r)`: the registry entry and version are live at the event epoch.
3. `Permitted(r)`: the principal and action are permitted by `Π_k`.
4. `Budget(r)`: declared sparsity, cost, and latency constraints hold where applicable.
5. `Parent(r)`: required parent/ancestry links are present and type-correct.
6. `Admissible(r)`: task-specific route requirements hold under a caller-declared oracle or rule, if such a rule is part of the contract.

Then

`A_route(k) := ∀r∈RequiredRoutes(k), Typed(r) ∧ Live(r) ∧ Permitted(r) ∧ Budget(r) ∧ Parent(r) ∧ Admissible(r).`

`Admissible` is optional only when explicitly declared out of scope; it is not silently `True`. `A_route` does not infer semantic expert quality from balance or gate magnitude. If `RequiredRoutes(k)` is empty because no routes were required, the checker returns vacuity metadata; Section 6.6 addresses vacuous passes.

4.3 Ledger predicate

Definition 19 (`A_ledger`). Define the following conjuncts over `T_k`:

- `PROV_k`: every accepted record has nonempty, typed, live provenance and content-bound source lineage where required.
- `AUTH_k`: every commit has `Act=1` under the action's authenticated source, hierarchy, and user-intent evidence.
- `TRANSITION_k`: each `L_{j+1}=δ(L_j,p_j)` satisfies schema, authority, temporal, and scope rules.
- `NOSILENT_k`: every replacement has a valid supersession link and the previous record remains queryable.
- `COV_k`: every required unit beneath level-`k` chunks has a valid content-bound receipt, with scope matched to the claim or route.
- `ORDER_k`: happened-before follows explicit dependencies, no receipt precedes intake, no commit precedes an authorized proposal, and no dependent read uses a future snapshot.
- `CORR_k`: every authorized applicable correction has reached all currently required dependents in `G`, or those dependents are explicitly quarantined from release.

Then

`A_ledger(k) := PROV_k ∧ AUTH_k ∧ TRANSITION_k ∧ NOSILENT_k ∧ COV_k ∧ ORDER_k ∧ CORR_k.`

`COV` and `CORR` are separated because complete intake does not show that later corrections propagated. `ORDER` and `AUTH` are separated because causal precedence does not authenticate an issuer. `PROV` and `COV` are separated because one cited source lineage does not establish coverage of all required units.

4.4 Generation predicate

Definition 20 (`A_gen`). For every in-scope generated claim `q` in `y_k`:

- `Lineage(q)`: each `c∈Cite(q)` has `Lin_k(c) ≠ ∅`, with no failed type cast.

- `ActiveEvidence(q)`: every required evidence index was activated in a snapshot visible to `q`.
- `Fidelity(q)`: `Support(q)=SUPPORTED`, all ancestors required for support are supported, and no ancestor is `UNSUPPORTED` or unquarantined `CONTESTED`.
- `StatusType(q)`: the claim's asserted status does not exceed source status; `{proved, strategy, analogy, measured}` are not interchangeable.
- `Scope(q)`: measured claims remain inside tested workflow scope and source validity intervals.
- `AttributionCompleteness(q)`: every claim class requiring attribution has at least one supporting citation, not merely a real-looking identifier.

`A_gen(k) := ∀q∈Claims_required(y_k), Lineage(q) ∧ ActiveEvidence(q) ∧ Fidelity(q) ∧ StatusType(q) ∧ Scope(q) ∧ AttributionCompleteness(q).`

The pointer-like condition `Cite(q)⊆ActiveIndices` is necessary for this restricted contract but insufficient for fidelity. A real document ID attached to a fabricated sentence fails `Fidelity`. An uncited common-knowledge stylistic claim is included only if the audit schema classifies it as externally dependent.

4.5 Continuity predicate

Definition 21 (`A_cont`). At scale `k`, require:

- `Projection_k`: `C_facts,k = π_{C←L}(View(L_k))` for the declared discrete projection, or every permitted abstraction has a reversible/explained summary lineage.
- `Obligation_k`: every in-scope open obligation remains open or has an authorized satisfaction/waiver witness.
- `Handoff_k`: every route change, agent handoff, session resume, and context compression emits the required continuity checkpoint.
- `CorrectionContinuity_k`: `C_corr,k` contains every applicable active correction and no quarantined dependent is presented as certified.
- `Directional_k`: every `d∈□_required,k` has `Covered_k(d)` when directional coverage is part of the contract.
- `NoAuthorityInflation_k`: continuity summaries and experts have not acquired commit authority beyond their declared role.

`A_cont(k)` is the conjunction of the required clauses. This predicate does not claim a universal scalar coherence metric. It asks whether declared finite obligations survived a specific transformation. A beautifully written continuation can fail `Projection`; an awkward one can pass.

4.6 Composite per-scale alignment

Definition 22 (`A(k)` and `ALIGNED_k`).

`A(k) := A_route(k) ∧ A_ledger(k) ∧ A_gen(k) ∧ A_cont(k).`

For the field's coverage/order/correction certificate, with optional routing stickiness,

`ALIGNED_k := COV_k ∧ ORDER_k ∧ CORR_k (∧ STICKY_k when required).`

The full certificate uses A(k). ALIGNED_k is a narrower coverage/order/correction diagnostic included within the ledger checks, with STICKY_k additionally required where routing uses it; it is not an alias for A(k). This avoids treating COV, ORDER, and CORR as sufficient for semantic generation or continuity.

Definition 23 (Global alignment). For scales 0,…,M,

ALIGNED(T) := ∀k∈{0,…,M}, A(k)=True.

Under three-valued internal checking, ALIGNED=False if any A(k)=False, and UNKNOWN if none is false but at least one is unknown. A CERTIFIED outcome additionally requires release authority and non-vacuity; it is not synonymous with raw ALIGNED.

4.7 First failure and trace

Definition 24 (Failure vector). Let b_k=(A_route(k),A_ledger(k),A_gen(k),A_cont(k)). The trace is 𝔅(T)=(b_0,…,b_M) together with sub-conjunct outcomes and witness references. Define

FIRST_FAIL(T) := min{k : A(k)=False},

when the set is nonempty. Separately, FIRST_NONPASS(T) := min{k : A(k) is UNKNOWN or NO_VERDICT, and A(j)=True for every j<k} when that set is nonempty. If a later observed failure exists, it does not erase an earlier non-pass; FIRST_FAIL and FIRST_NONPASS are reported as distinct locations rather than overriding one another. FIRST_FAIL is unique because it is the minimum of a finite set. This is a definitional fact, not a Kakeya theorem.

A certificate includes the first failure **and** all later observed failures. Later checks may be unsafe to execute after an earlier failure; those are marked SKIPPED, not True. Thus early failure does not erase the rest of the evidence.

4.8 Two-ends support condition

Definition 25 (TWO_ENDS). For claim q, let SupportUnits(q) be the source units in its supporting ancestry and let ε>0, θ_2E be caller-supplied. Then

TWO_ENDS(q) := no admissible window shorter than ε|S| contains more than a θ_2E share of q's declared support.

The window family and support weights must be specified; otherwise the result is UNKNOWN. This condition is inspired by two-ends vocabulary, not imported as a theorem. It detects support concentration that a receipt count can miss. It is optional for local single-source extraction and mandatory only where the evaluation contract requires distributed evidence.

4.9 Non-compensatory semantics

A score 0.99 on three predicate families does not compensate for AUTH=0. Formally, the release lattice is conjunctive: replacing any required True by False makes A(k) false regardless of all other values. Scalar telemetry can still summarize defect density,

witness latency, or route entropy, but cannot reverse a certificate outcome. This is the paper's operational interpretation of “boolean alignment”: named necessary conditions, not a weighted sense of aligned-ness.

5. Induction on Declared Scales

The schedule below is a systems methodology. The propositions about finite trees, graphs, and lists are proved inside the stated model. Induction runs over declared abstraction levels, not automatically over model size, training compute, token count, or an external geometric scale.

5.1 Obligations, semantic predicates, and evidence predicates

At scale `k`, let `Q_k` be the independently specified finite obligation universe. An obligation can concern one expert activation, one retrieval, one ledger transition, one citation, or a **cross-operation invariant** such as a shared capacity or correction frontier. For obligation `q`, fixed history `H_q`, policy snapshot `Π(q)`, and specification `Spec_k`, semantic authorization and compliance predicates are

`A_q(H_q,Π(q))∈{0,1}`, `C_q(H_q,Spec_k)∈{0,1}`.

They answer different questions. An authorized write can violate a schema; a well-formed retrieved instruction can be unauthorized. A monitor with evidence `E_q` returns `V^A_q,V^C_q∈{T,F,U}`, where `U` covers both missing evidence (`UNKNOWN`) and missing contract (`NO_VERDICT`) at the diagnostic layer. A **sound verdict** satisfies

`V^A_q=T ⇒ A_q=1`, `V^A_q=F ⇒ A_q=0`, and similarly for `C_q`.

This is a requirement on the checker, not a property conferred by its name. Independent specification review, negative controls, and adversarial review are needed to assess it. The monitor's self-score cannot be its ground truth. `FOUND` is evidence of retrieval, never an authorization bit. A test-runner crash is not a semantic `F`.

5.2 Grains and slabs as typed audit records

We use **grain** only as a structural analogy for an inspectable packet:

```
g = (id, scale, source_identity, span, content_hash, status,
     authority_stamp, dependencies, governing_ack[], transfer_receipts[])
```

`content_hash` binds intake bytes; it is not a read receipt, semantic support, freshness, or permission. `status` distinguishes proved, proposed, strategy, analogy, measured, and observed. `authority_stamp` belongs to a channel external to content. A **slab** is a bounded collection of grain records at one scale with a declared partition rule, overlap rule, and typed cross-scale morphisms. It is not a geometric slab. Paper grains, SAVER grains, MoE experts, and source chunks are not identified with one another.

For every applicable ancestor `a` of grain `g`, `ANCEST_OK_k` requires an acknowledgment `(a.id,c_id,c_ver)` in `g.governing_ack`, where `(c_id,c_ver)` is the currently governing correction in `a`'s authoritative registry and the acknowledgment is authorized under `a`'s channel. An authorized supersession may replace it. `NEST_OK_k` requires fresh parent edges and content hashes. Then

```
STICKY_k := NEST_OK_k ∧ ANCEST_OK_k ∧ optional declared clustering caps.
```

No global `correction_version(parent)≤correction_version(child)` comparison is used: counters from different registries are incomparable. This **stable-custody nesting** is neither the sticky case of an external Kakeya proof nor KNS fan closure.

5.3 Sticky case: unique visible parentage

The simplest audit case has a unique coarse parent for each fine item, shared authority and correction scope, and transfer receipts that make the coarse checker depend only on its declared children. Let `D_k(q)⊆Q_{k-1}` be the child obligations needed for parent obligation `q`. In the sticky case, every required child has one visible parent, all source versions agree with the snapshot, and no unlisted execution channel affects the parent.

A sticky pass is not a semantic theorem. It gives a small certificate: parent identity, children, content-bound receipts, correction acknowledgments, and a refinement witness. If any child has multiple incompatible authorities, missing acknowledgment, divergent snapshot, or cross-parent dependency, the trace leaves the sticky case. This is a change in audit regime, not a geometric singularity.

5.4 R: evidence-preserving refinement

Refinement condition R. For each parent obligation `q`, there is an abstraction map and child set `D_k(q)` such that

```
(∧_{r∈D_k(q)} (A_r∧C_r)) ⇒ (A_q∧C_q).
```

The child certificates and refinement witness must establish the implication without enlarging authority or scope. A real refinement records how child operations implement the parent and includes every cross-child premise. Renaming an index, matching a display label, or observing local fluent output is insufficient. Finer-scale `PASS` does not imply coarser semantic fidelity, and coarse `PASS` does not imply every finer obligation; `CROSS_SCALE_OK` names the required transfer receipts.

5.5 C: complete obligation coverage

Coverage condition C. `D_k(q)` accounts for every operation, effect, and cross-operation premise relevant to `q`, and `Q_k` is closed under work generated during the trace. Retries, asynchronous callbacks, dynamically created tasks, and side effects must extend the universe. Coverage is not the fraction of conveniently logged calls.

Independent evidence must establish the obligation universe. “All five obligations are present” cannot itself prove that five are all required; that would be circular. A specification manifest, externally reviewed task model, or independently enumerated source inventory supplies the denominator. If a tolerated uncovered fraction is used, it

may support a risk statement but not the exact global boolean proposition below. Missing `min_coverage`, required thresholds, or a nonempty applicable scale family yields `NO_VERDICT`.

5.6 N: noninterference and authority isolation

Noninterference condition N. Operations outside the certified obligation set cannot alter state on which authorization, compliance, evidence, or refinement depends. Unverified content cannot rewrite policy, registry, authority, snapshots, or verifier output. Routers propose but do not commit; generators propose ledger entries but do not self-accept; imported ledgers are rechecked under importer authority; child success cannot enlarge child scope.

Append-only storage does not supply N. It can faithfully preserve an unauthorized write. Similarly, immutable evidence can preserve a stale approval. Capability isolation is a semantic restriction on influence, not just a diagram with separate boxes.

5.7 K: correction-aware concurrency and freshness

Condition K requires: (1) policy, registry, authority, and relevant ledger reads come from a stated snapshot with linearizable or equivalently serializable commit semantics; (2) every causally preceding correction is included, without fabricating concurrent corrections into dependencies; (3) versions remain valid at effect boundaries or are revalidated; (4) accepted corrections transitively invalidate dependents before promotion; (5) conflicting concurrent operations are ordered or explicitly resolved; and (6) replayed approvals and nonces do not create fresh authority.

Logical clocks help represent visibility, but Lamport order is not authority or consensus. A historically valid signature is not necessarily current. `NOT_YET`, `ABSENT`, `LOST`, and `FOUND` must remain distinct: causal invisibility is not nonexistence, failed retrieval is not a legitimate negative, and retrieval is not permission.

5.8 Non-sticky band factoring and affected closure

A non-sticky band is a set of grains, claims, and transitions for which sticky nesting or another structural predicate is non-pass. Repair proceeds by **affected dependency closure**, not by editing only the visible seed:

1. Identify failing or unresolved seeds `Z_k`.
2. Compute `Aff = Reach_G(Z_k)`, including transitive dependents across indices, scales, generated claims, ledger projections, and pending actions.
3. Mark every item in `Aff` stale or quarantined, with `UNKNOWN`, `FAIL`, or `NO_VERDICT` according to evidence; do not erase history.
4. Retain evidence outside `Aff` unchanged if N holds and its own freshness remains valid.
5. Repair, re-receipt, re-acknowledge, or reauthorize within the band and along every affected edge.
6. Recheck all of `Aff`, including cross-scale transfer, rather than just the seed.

7. Stop after a caller-supplied `max_rounds`; exhaustion gives `NO_VERDICT` or a correction-storm refusal, never an automatic pass.

Local repair does not imply semantic stability outside the rechecked closure. If N fails, the assumption that outside evidence is unaffected is itself unjustified and the closure must expand.

5.9 Capped general schedule

The proposed schedule is:

```
for k = 0 … M:
    establish independent Q_k and policy/registry/snapshot versions
    if sticky contract holds:
        verify local obligations and refinement witnesses
    else:
        factor only the non-sticky band
        compute and repair Aff under R, C, N, K
    recheck all affected descendants and cross-scale receipts
    record residuals and non-PASS statuses
return STRUCTURAL_READY only if every required level is PASS
```

A finite loop is not by itself an induction proof. The proof uses explicit child-to-parent implications and complete coverage. `max_rounds` bounds reconciliation; it does not establish correctness. Residuals remain visible in the certificate.

5.10 Structural readiness versus full release

Define

```
STRUCTURAL_READY_k = b(COV_k) ∧ b(ORDER_k) ∧ b(CORR_k)
                   ∧ b(ROUTE_OK_k) ∧ b(LEDGER_OK_k) ∧ b(GEN_OK_k)
                   ∧ b(A_cont,k) ∧ (b(STICKY_k) if required).
```

`STRUCTURAL_READY` requires every level to pass either directly or after capped reconciliation and affected-closure recheck. It is an operational structural surrogate, not measured semantic Index Alignment and not `SUPPORT/FID`.

Full promotion separately requires

```
RELEASE_FULL = b(STRUCTURAL_READY) ∧ b(CROSS_OK) ∧ b(CROSS_SCALE_OK)
             ∧ b(SCOPE when measured claims occur) ∧ b(POLICY_OK)
             ∧ b(AUTH) ∧ b(SUPPORT) ∧ b(FID) ∧ b(NONVACUOUS).
```

`POLICY_OK` and `AUTH` use evidence independent of ingestion receipts. `SUPPORT/FID` use a declared external assessor contract whose reliability is unresolved unless independently validated. No receipt can certify semantic truth. Unknown or no-verdict evidence blocks full promotion but is not automatically a semantic failure; authorized unaffected partial output remains possible.

---

6. Conditional Propositions and Failure Localization

6.1 Sound three-valued aggregation

Proposition 1. Let Q be finite. For each $q \in Q$, let $V_q \in \{T,F,U\}$ be a sound verdict for semantic predicate $P_q \in \{0,1\}$. Let $G = \bigwedge_q P_q$ and let $V_G = \bigwedge_q V_q$ under strong Kleene conjunction: F dominates, T is identity, and otherwise the result is U. Then $V_G = T \Rightarrow G = 1$, $V_G = F \Rightarrow G = 0$, and if no $V_q = F$ but some necessary $V_q = U$, neither semantic value is certified.

Proof. If $V_G = T$, every $V_q = T$; soundness gives every $P_q = 1$. If $V_G = F$, some $V_q = F$; soundness gives $P_q = 0$, so the semantic conjunction is false. In the remaining case, no false conjunct determines the aggregate and at least one unknown blocks a true monitor verdict. U asserts neither semantic truth nor semantic falsity. ∎

This result distinguishes a semantic boolean from monitor knowledge. It does not make an unsound checker sound. `NO_VERDICT` is an operational presentation of U, not a fifth truth value. A boolean release projection $b = 1$ only on PASS is a control rule, not a claim that unknown evidence is false.

6.2 Conditional induction across scales

Proposition 2. Fix finite scales $0, \ldots, m$ and finite traces. Assume: (1) every obligation in Q_0 has sound T verdicts for authorization and compliance; (2) R holds at every adjacent scale; (3) C holds there; (4) N holds throughout; and (5) K holds throughout. Then every parent obligation at every scale has $A_q = C_q = 1$. If the global specification is the conjunction of obligations at scale m, global authorization and compliance are true. Every global T verdict has a finite certificate path to base evidence.

Proof. The base case is assumption 1 and verdict soundness. Assume all obligations at scale $k-1$ have true authorization and compliance predicates with sound certificates. Choose $q \in Q_k$. By C, $D_k(q)$ accounts for all operations, effects, and cross-operation premises relevant to q. By N, no outside operation changes those predicates or their evidence, and composition grants no new authority. By K, certificates use current correction-closed snapshots and stale dependents cannot survive promotion. The induction hypothesis makes every predicate in $D_k(q)$ true. R yields $A_q = C_q = 1$. Appending the refinement witness to child paths yields the parent witness. Finite induction proves the claim. ∎

The hypotheses are substantial. Real systems have not been shown to satisfy them. The proposition does not imply that every desirable global property is conjunctively decomposable, that model scaling preserves alignment, or that a lineage certificate proves entailment. It is a proof about a formal model, not an empirical theorem about deployed LLMs.

6.3 Existence and uniqueness of the first observed failed scale

Proposition 3. For a finite total scale order, if $F = \{k : A(k) = \text{False}\}$ is nonempty, $\text{FIRST_FAIL}(T) = \min F$ exists and is unique. If sub-conjuncts at that scale are ordered by a declared rule, the reported first failed predicate is also unique.

Proof. A nonempty finite subset of a totally ordered set has a unique minimum. The same fact applies to the finite ordered list of failed sub-conjuncts at that scale. ∎

This says nothing about a unique semantic root cause. Several defects can cause the same failed predicate; an earlier unobserved cause may lie outside the trace. `FIRST_FAIL` is the earliest **known failed declared lift**, not a metaphysical origin.

6.4 Concurrent failures on DAGs

Definition 26 (Concurrent failure frontier). Let failed scale nodes form a finite set `F` in a DAG with declared causal order `≺`. Define `Frontier(F)={x∈F : no y∈F has y≺x}`. If `≺` is a valid topological partial order, return `Frontier(F)` and all failures incomparable with its members as a set. If required order edges are missing, return `UNKNOWN_ORDER`.

Proposition 4. Every finite nonempty `F` has at least one minimal failed node when `≺` is acyclic, and every failed node has a minimal failed ancestor in `F`.

Proof. Choose any `x∈F` and follow strict failed predecessor edges while possible. Acyclicity forbids repetition and finiteness forces termination at a minimal failed node. For arbitrary `x`, either it is minimal or this descent supplies a minimal ancestor. ∎

A single minimum need not exist: two independent failures can be incomparable. Certificates therefore return a set rather than arbitrarily choosing a “first” failure. A topological display order is only a presentation convention.

6.5 Finite computability of localization

Proposition 5. Suppose each declared predicate checker terminates on its finite slice, graph reachability is over a finite graph, and all required thresholds are present. Then the boolean trace, `Frontier(F)`, and affected closure are computable in finite time.

Proof. There are finitely many scales and obligations. Evaluate each terminating checker and record its result. Finite graph traversal marks reachable affected nodes in finite steps. The finite failed-node frontier is obtained by checking predecessor membership. Sorting or displaying the trace also terminates. ∎

This is not a polynomial bound for semantic discovery, independent source enumeration, or finding repairs. An LLM support assessor may be expensive, unreliable, or nonterminating in practice; timeouts become execution statuses, not semantic failures.

6.6 Non-vacuity requirement

Definition 27 (Vacuous certificate). A scale certificate is vacuous when its required obligation set is empty, its source denominator is unknown, all route or claim sets are empty despite required work, or its checks are skipped while reported as passes. An explicitly empty task with a proof that no obligations apply is different from an empty log.

Proposition 6. If the release contract requires `NONVACUOUS` and `NONVACUOUS` is not PASS, then `b(RELEASE_FULL)=0`.

Proof. RELEASE_FULL is a conjunction with PASS-only projection. Any non-PASS required conjunct has b=0, forcing the conjunction to zero. ∎

Evidence coverage is |Q_sound|/|Q_req|. If Q_req=∅, report n/a, not 100%. The empty denominator cannot be used to certify all work.

6.7 Conditional unaffected-content preservation

Proposition 7. Let correction seeds be Z, Aff=Reach_G(Z), and output set O satisfy Deps(O)∩Aff=∅. Assume N, stable evidence versions for O, and a complete dependency graph for the release scope. Then a correction confined to Z and its outgoing dependencies does not change the semantic predicates certified for O.

Proof. Every dependency of O lies outside Aff. By definition of reachability, no dependency path from Z reaches those dependencies. N excludes unmodeled influence from the correction or its execution. Stable evidence versions preserve the predicates' inputs; hence their values are unchanged. ∎

This justifies authorized partial release only when its own policy, authority, scope, and support checks pass. If dependency enumeration or N is incomplete, unaffectedness is unknown. Previously emitted unretractable copies cannot be recalled by graph invalidation.

6.8 Conditional retention on a finite DAG

AIce supplies an elementary retention proposition adapted here. For each node v, let Required(v) be the governing evidence and transitions needed for its current action and C(v) the admitted carried set. Assume base coverage of every reachable transition, edge preservation, admission of newly governing patches before action, valid scoped fresh supersession, an enforced gate on every execution, and no unmodeled execution channel.

Proposition 8. Under these hypotheses, topological processing gives Required(v)⊆C(v) for every reachable v.

Proof. At a source node, base coverage includes all required transitions. Suppose the claim holds for every predecessor. Edge preservation carries their required records into v; admission adds any newly governing patch before v acts; valid supersession replaces obsolete records without dropping current requirements. Thus the required set is contained in the carried set. Topological induction covers the finite DAG. The enforced gate ensures an execution cannot bypass this admission condition. ∎

The result is structural retention, not semantic sufficiency. Distinguish selected_action, gate_verdict, executed_action, and observed_effect: the shipped AIce simulator is advisory and can execute a rejected selection. A local enforcing wrapper blocks it, but a reported E2 false-stop shows enforcement can reject legitimate work. Custody is not enforcement.

6.9 Refinement invalidation

Proposition 9. If any certificate on the path from base evidence to parent `q` is invalidated, or R, C, N, or K ceases to hold on that path, the old parent `T` certificate is no longer justified by Proposition 2.

Proof. Proposition 2's parent witness consists of valid child paths and all composition hypotheses on the path. Removing any premise invalidates that instance of the proof. This does not establish that the parent semantic predicate is false; it establishes that the old certificate must be withdrawn or rechecked. ∎

This is why policy drift, source version changes, correction acknowledgments, and verifier updates are part of certificate identity.

6.10 Bounded verification proposition

Proposition 10. Given supplied certificates and an explicit finite obligation universe, let `n` be records, `|E|` dependency edges, `c` claims, `r` receipts, and `w` declared windows. Further restrict the witness so that dependency/support markings and the affected-closure union are checkable in one shared graph traversal, any additional lineage path is already counted in `n`, every declared window check is constant-time, and each claim triggers at most one support-oracle call. If primitive schema, signature, hash, and threshold checks cost constant time and each support oracle call costs `O(B)`, verification under these restrictions costs

```
O(n+|E|+cB+r+w)
```

plus the cost of sorting where canonical display order is requested. If support text length is bounded, this is linear in supplied records and edges.

Proof. One pass checks schemas and local predicates. By the additional restriction, shared BFS/DFS and support/dependency propagation visit each supplied graph node and edge once rather than repeating traversal per claim or per seed. Claim checks make at most `c` support calls, receipt checks at most `r`, and constant-time window checks at most `w`; supplied lineage material is included in `n`. Sorting adds its standard comparison cost. Without these restrictions, repeated per-claim or per-seed traversals must be charged separately. ∎

The bound applies to a **supplied restricted witness**, not to discovering obligations, searching for a good route, training a model, or proving semantic support. Cryptographic verification and large integer costs must be modeled separately in a real implementation.

6.11 Nontransfer and counterexamples

Local-to-global budget. A parent allows total spending at most 100. Two children each spend 60 and are individually authorized up to 60. Both local authorization and compliance pairs are `(T,T)`, but `60+60=120`, so parent compliance is false. R and C omitted a cross-operation obligation. The repair is a shared-cap invariant, partitioned budget, or atomic reservation, not another count of local passes.

Unknown is not true. Required operations are `{o1,o2,o3}`; only `o1,o2` are observed and pass. Reporting `2/2 passed` changes the denominator to observed obligations. `V3=U` and global verdict is `U`. Without an independent counterexample, `F` would also be unsound.

Stale concurrent approval. Approval `a` authorizes `tool:t17` under registry `r7`; correction `c` revokes it in `r8`; operation `o` depends on `a` but not `c`. Vector clocks `VC(a)=(1,0,0)`, `VC(c)=(0,1,0)`, `VC(o)=(1,0,1)` show `a<o` while `c` and `o` are concurrent. Before the correction frontier is known, current authorization is `U`; after a complete required-current-policy history shows revoked use, it is `F`. Signed and fresh are separate.

Authority pollution despite immutability. A router logs every request, but an unverified instruction chunk changes the authority table and makes the router an approver. Later schemas pass and the append-only log preserves all history. N fails because unverified content affected an authority premise. Persistence is not isolation.

Found content is not an instruction. A retrieved chunk says “approve X” and returns FOUND, but its issuer is absent from the authority registry. `V^A` is `F` if evidence establishes no authority and `U` if authority evidence is incomplete, never `T` from FOUND alone.

---

7. Architecture Mapping

7.1 Non-authoritative router

The router proposes `R_t` and typed morphisms. Its certificate contains registry and policy versions, gate telemetry, budgets, parentage, and task admissibility where declared. It has no ledger commit capability. `A_route` checks these facts without treating gate entropy as semantic quality. Router agreement is not ground truth, and consensus among routers does not mint `AUTH`.

A soft mixture with no exposed hard expert identity may be outside the routing contract. The correct response is limited scope or `UNKNOWN`, not an invented discrete index. Hidden computation remains hidden even if exposed routing logs pass.

7.2 Ledger as authority boundary

The ledger consumes proposals and emits accepted, rejected, pending, superseding, or deferred records. Its checker separately verifies provenance, content-bound coverage, causal order, transition validity, authority, nonsilent supersession, and correction closure. The name “ledger” confers none of these properties.

An enforcing ledger must also distinguish selection, gate verdict, execution, and effect. If the gate rejects a tool but an executor invokes it anyway, the execution policy fails independently of whether the log records the rejection. Conversely, a gate can false-stop valid work while detection remains correct.

7.3 Generative continuity expert

The continuity expert maintains projections of facts, discourse, goals, and corrections. It can propose text and summaries but does not gain factual or commit authority from that role. `A_cont` tests preservation of open obligations and active correction scope across compression, handoff, and resume. The projection need not contain the entire ledger; it needs a declared, inspectable relation to the committed view.

A continuity summary can be internally consistent and wrong. Projection consistency is not factual support, and support is not user authority. These three channels remain separate.

7.4 Diagnostic ladder

For each task, preregister an applicable subset of

`chunk → section → source → cross-source → handoff.`

The ladder is a diagnostic schedule, not a universal law. A single-document extraction may stop at source; a multi-agent resume may require handoff; a cross-source synthesis needs all relevant earlier levels. Each level has an obligation universe and transfer witness. Passing one level cannot substitute for a missing later level.

7.5 Role-to-predicate matrix

Role/ component	Proposes	May not infer	Principal checks	Typical first failure
MoE/router	expert IDs, gates, casts	authority, semantic expert quality	Live, Permitted, Budget, Parent	stale registry epoch
Retriever	document/chunk indices	support, reading, permission	lineage, COV, scope	inactive citation
Tool gateway	call proposal	success from selection	AUTH, Π, input schema	revoked authority
State ledger	accepted view	truth from append order	TRANSITION, NOSILENT, ORDER	silent clobber
Generator	claims and citations	factual support from IDs	Fidelity, StatusType, Scope	unsupported claim
Continuity expert	summary and open goals	new commit authority	Projection, Obligation, Handoff	dropped obligation
Release controller	certificate outcome	promotion from coverage	CROSS, CROSS_SCALE, SUPPORT/FID	unknown assessor

7.6 Certificate governance manifest

The manifest versions criteria as well as code. It names specification, registry, policy, authority, source inventory, thresholds, support assessor, mutation set, and release scope. Criteria drift is a new certificate epoch, not a silent edit to old results. EvalGen's criteria-drift findings motivate this versioning; they do not validate this design.

Independent judgments must be collected before discussion where human or model assessors are used. Consensus after discussion can hide correlated anchoring. Sycophancy and BenchForm results motivate this protocol but supply no reliability estimate here.

---

8. Correction, Invalidation, and Partial Release

8.1 Corrections are scoped graph operations

A correction changes the status of evidence or commitments only when its issuer, applicability, scope, and freshness are established. `Authorized(c)` is independent of whether many agents repeat it. Once accepted, the correction seed `target(c)` identifies the start of `Dependents(c)`, computed over claim, ledger, routing, and output dependencies. All dependent certificates become stale before any new promotion.

The operation preserves history. The prior value remains queryable, the supersession edge records why it changed, and dependent certificates retain their old evidence with an `INVALIDATED` marker. Deleting the old record would destroy the ability to audit whether correction propagation worked. A valid correction can itself later be corrected; each new version forms a new frontier.

8.2 Freshness and invalidation table

Predicate	Required freshness evidence	Event that invalidates old certificate
COV	challenged span, source version, nonce/window	version bump or span mismatch
ORDER	explicit edge-set version	rewrite without supersession
CORR	governing correction IDs and frontier	new applicable authorized correction
ROUTE_OK	registry and policy TTL	deregistration or policy revocation
LEDGER_OK	ledger snapshot and acknowledgment set	forged write or stale acknowledgment
GEN_OK	activation and citation snapshot	route changes without regenerated lineage
CROSS_SCALE_OK	transfer receipt versions	affected ancestor or missing transfer
SCOPE	tested-workflow registry	claim leaves measured scope
POLICY_OK/AUTH	policy and authority channel versions	revocation or expiry
SUPPORT/FID	external assessor run and rubric	assessor unavailable, drift, or scope change

A preserved artifact can be obsolete; custody is not freshness. An assessor `UNKNOWN` is not evidence that a claim is false, but it blocks a PASS-only release bit.

8.3 Five certificate outcomes

`CERTIFIED` requires all full-release conjuncts to pass, independent release authority, and non-vacuity. `HELD_PARTIAL` emits only authorized unaffected content and names held dependencies. `REFUSED` records an observed failure that prohibits the requested release. `AWAITING_HUMAN` is used when an authorized adjudicator is required; it does not mean a human is universally necessary. `NO_VERDICT` records missing thresholds, incomplete evidence, unavailable assessor, exhausted reconciliation, or invalid execution.

The same predicate trace can therefore lead to different control outcomes under different authorized scopes. A factual failure in one section can coexist with an unaffected partial extraction. Conversely, a semantic pass in a sandbox can coexist with refusal to publish because `AUTH` is absent.

8.4 Partial release rule

For candidate output `o`, partial emission requires

```
Deps(o) ∩ Aff = ∅
∧ o's route, authority, policy, support, scope, and continuity checks PASS
∧ dependency enumeration and N are PASS for its scope
∧ output is labeled PARTIAL and names excluded obligations.
```

If any of these is unknown, `o` is not certified unaffected. `Act=0⇏Stop` means a privileged action may be denied while a labeled informational response is still emitted. It does not override a separate policy requiring complete suppression.

8.5 Independent verifier boundary

PROV-002 requires a verifier outside the process capable of forging receipts. A challenged quotation generated and checked by the same process is an honest-workflow control, not a security boundary. An in-process HMAC does not protect against code holding the same key. Independent review includes source access, specification review, and named negative controls; model agreement alone is not independence.

---

9. Worked Boolean Traces

All traces below are **hand-constructed synthetic illustrations**. No external action was performed, no model was run, and no empirical success rate is implied. `T/F/U` abbreviate `PASS/FAIL/UNKNOWN`; `NV` denotes missing contract. Execution outcomes are listed separately.

9.1 Trace A: valid sticky two-level retrieval lift

Policy `P9`, registry `R4`, authority `Auth3`, snapshot `S12`, and `min_coverage=1.0` are fixed. An independent specification manifest lists exactly five obligations: route `w1` to `d3`; read `d3` with verifier challenge `q88`; propose `m:/plan/x` with valid signature and unused nonce; generate a claim with lineage to both indices; and check the parent coverage/continuity invariant. The fifth obligation's completeness comes from the manifest, not from counting the five rows in this table.

Scale	Predicate evidence	Outcome
0 routing	d3 live in R4, w1 permitted	A_route=T
0 evidence	quoted span matches independent verifier copy	COV=T
0 ledger	signature under Auth3, schema and nonce valid	A_ledger=T
0 generation	citations resolve to active indices; external support check passes	A_gen=T
0 parent invariant	no outside effect, projection matches, correction frontier complete	A_cont=T
1 lift	typed identities preserved, only o3 changes state, R/C/N/K witnesses pass	A(1)=T

The resulting `STRUCTURAL_READY=T`; if `CROSS_SCALE_OK`, `AUTH`, `POLICY_OK`, `SUPPORT/FID`, and `SCOPE` also pass, the restricted certificate is `CERTIFIED`. The illustration still does not prove that natural-language truth follows from citation membership: support is an explicit additional assessor input.

9.2 Trace B: registered but unauthorized MoE substitution

At a token-routing layer, `expert:e7` is live, typed, and within budget. Registry policy permits code analysis only for `e8`, but the router activates `e7` for a privileged repository transformation. Gate weights and load balance are normal.

`Typed=T`, `Live=T`, `Budget=T`, `Permitted=F`, hence `A_route=F`. Other predicates may be evaluated for diagnosis but cannot compensate. `FIRST_FAIL=0 (A_route.Permitted)`. Execution is `REFUSED` for the privileged transformation. A non-privileged explanation of the routing mismatch may be emitted if separately authorized. Registry membership alone did not confer authority.

9.3 Trace C: retrieval coverage with false fidelity

Every required source chunk has a fresh content-bound receipt, causal order is valid, and all citations point to active document IDs. The generated conclusion says a medication dose was doubled, while the source says it was halved. The external support assessor identifies the contradiction.

`COV=T`, `ORDER=T`, `Lineage=T`, `ActiveEvidence=T`, `Fidelity=F`; therefore `A_gen=F`. This trace is the canonical coverage-pass/fidelity-fail counterexample. Full release is refused even though all receipt counters are green. It also demonstrates why `GEN_OK` defined merely as active citation lineage is only structural and cannot replace `SUPPORT/FID`.

9.4 Trace D: long-context middle omission

A synthesis task requires evidence at beginning, middle, and end of a long input. Beginning and end chunks have challenged quotations. The middle chunk has only a file hash; no quoted span or independent coverage receipt is present. Two generated claims are supported, and the third may or may not be true.

`COV_middle=U`, not `F`; `A(1)=U` if no observed predicate fails. The release bit is zero. Lost-in-the-middle results motivate testing this location, but this trace does not assume every model omits the middle. The certificate reports missing evidence rather than claiming an observed semantic omission. Authorized beginning/end extracts may be partial only if their dependencies exclude the unresolved middle claim.

9.5 Trace E: stale tool authority at commit

At proposal time, signed approval authorizes `tool:t17` under registry `r7`. Before execution, an applicable correction revokes it in `r8`. The tool call's causal record includes the old approval but not the correction; the correction frontier at commit is known and requires current registry validity.

The historical signature check passes. `AUTH_current=F`, `ROUTE_OK=F`, and the affected output is quarantined. If the frontier were not yet known, the monitor outcome would be `U`, not `T`. This distinguishes stale evidence from fabricated evidence: the approval was genuine but obsolete.

9.6 Trace F: concurrent agent failures

Agent A commits a valid plan; Agent B uses a revoked memory item; Agent C drops an unrelated open obligation. B and C have no causal order between their failures, and both occur after A. The DAG failed set has two minimal failed nodes.

`Frontier(F)={B.A_ledger.CORR, C.A_cont.Obligation}`. No unique `FIRST_FAIL` is reported under the causal partial order. A topological display may show B first, but the certificate preserves incomparability. Repair computes two affected closures and their union for shared outputs. Partial release of A is allowed only if its dependencies intersect neither closure and its own release checks pass.

9.7 Trace G: correction propagation and unaffected partial

A source correction changes `d4` from version 2 to 3. Claim `q4`, summary `s4`, and pending tool action `x4` depend on `d4`; unrelated extraction `q1` depends only on `d1`. Independent dependency enumeration and N pass.

`Aff={d4,q4,s4,x4}`. Their certificates are invalidated, while `q1`'s evidence remains stable. `q4/s4/x4` are held; `q1` can be `HELD_PARTIAL` with an explicit exclusion notice after its own authority and support pass. If `q1` had been emitted in an earlier irreversible external message, graph invalidation would not retract it; this protocol addresses future promotion, not recall of all copies.

9.8 Trace H: correction storm and round cap

Two authorized registries alternately supersede conflicting values for the same claim. Each correction correctly invalidates the other branch's dependent certificates. The caller sets `max_rounds=3`; no stable correction-closed snapshot emerges by the cap.

The structural predicates are not declared true. Outcome is `NO_VERDICT` with reason `CORRECTION_STORM`, or `REFUSED` if policy expressly prohibits unresolved contested output. No arithmetic average of versions chooses a winner. Authority, applicability, and conflict resolution must be adjudicated outside the loop; `AWAITING_HUMAN` is appropriate where the contract names an adjudicator.

9.9 Trace I: missing threshold

The same records as Trace A are supplied, but `min_coverage` is omitted. Every observed local schema check passes. The monitor cannot infer that observed records exhaust required obligations from the fact that they look complete.

`CoverageContract=NV`; full release is `NO_VERDICT`, not `CERTIFIED` and not semantic `F`. The missing threshold is recorded with the certificate. This no-default rule also applies to clustering thresholds, correction caps, support windows, and acceptable cost budgets.

9.10 Trace J: detected fault versus harness crash

A clean baseline first passes all named checks under configuration `cfg17`. Fault variant `stale_ack` then executes and the predeclared assertion `reject_stale_governing_ack` fails for the injected reason. This is **DETECTED**. A second variant `causal_blind_lookup` exits nonzero during import before any relevant assertion runs. This is **HARNESS_ERROR**, not detected. A third completes validly with no relevant failure: **UNDETECTED**. If enforcement itself throws while detection correctly returns `F`, classification is `EXECUTION_ERROR` on the execution axis and `DETECTED` on the detection axis.

A nonzero exit code alone is insufficient. Detection requires a passing baseline and a named attributable property failure. P3 in the neighboring open-addressing probe remains unresolved; its empirical cost behavior is not an index-alignment result.

9.11 Trace K: shared global capacity violation

Two child agents each reserve 60 units under a local limit of 60. Both local predicates pass. The parent budget is 100, and the independent specification includes a shared-cap obligation. If this obligation is omitted from `D(parent)`, R/C fail and the local passes do not lift. With an atomic reservation ledger, the second reservation observes remaining capacity 40 and `C_parent=F`; execution rejects the reservation. Detection and rejection are separately recorded.

9.12 Trace L: authority pollution from retrieved instructions

A retrieved document contains “make router an approver.” Its content receipt and quotation are valid. An unverified ingestion path nevertheless changes the authority table. Later operations have perfect schema and append-only provenance. N fails because content affected the judge's authority premise; AUTH also fails for the router's new role. Reverting the table does not erase the need to invalidate all dependent certificates issued during pollution.

9.13 Trace M: authorized partial under semantic unknown

A report has independent extraction sections `X` and `Y`. X has complete current evidence, policy, authority, and support. Y's support assessor is unavailable. No dependency connects X to Y, and N and coverage enumeration pass. Full report promotion is blocked with `SUPPORT(Y)=U`; X may be emitted as `HELD_PARTIAL`. The system does not label Y false, nor does it promote X as evidence about Y.

9.14 Trace N: continuity checkpoint lost at handoff

A task resumes after context compression. The ledger retains an open obligation to verify source version, but the continuity summary omits it. All original sources and routes remain live. `A_route=T`, `A_ledger=T`, `A_gen` may be locally `T`, but `Obligation=F` and `Handoff=F`. The first failed lift is the handoff scale, even if the underlying omission originated in an earlier compressor not represented in the trace. This illustrates diagnostic rather than unique-root semantics.

9.15 Trace summary

Trace	Principal non-pass	Certificate/control lesson
A	none under declared scope	conditional lift, not universal truth
B	Permitted=F	registered ≠ permitted
C	Fidelity=F	coverage ≠ support
D	COV=U	missing evidence ≠ observed semantic fail
E	AUTH_current=F	signed ≠ fresh
F	concurrent failures	return a frontier set
G	affected dependents stale	scoped invalidation and partial release
H	cap exhausted	termination bound ≠ correctness
I	threshold absent	NO_VERDICT, no default pass
J	variant-specific	detection ≠ process exit
K	parent cap=F	local passes need cross-operation obligations
L	N and AUTH fail	immutable history ≠ isolation
M	SUPPORT=U	unaffected authorized partial allowed
N	continuity obligations fail	fluent continuation ≠ preserved commitments

---

10. `INDEX-ALIGN-CERT`: Decision Object and Complexity Boundary

10.1 Input and witness

Definition 28 (Decision object). An instance of `INDEX-ALIGN-CERT` contains:

1. finite typed index spaces and registry epochs;
2. a finite trace with explicit causal edges and snapshots;
3. independently declared scale tree and obligation universes;
4. policy, authority, and user-intent evidence for requested actions;
5. claim and dependency graphs, content receipts, corrections, and transfer witnesses;
6. fixed specification versions and caller thresholds;
7. verifier contracts, including support/fidelity assessors and non-vacuity rules;
8. release scope and `max_rounds`.

A witness is the vector of predicate outcomes, evidence and counterexample references, refinement paths, affected closures, residual items, and evidence supporting the release decision. The release decision itself is verifier output, not evidence of its own correctness. The output set is `{CERTIFIED, HELD_PARTIAL, REFUSED, AWAITING_HUMAN, NO_VERDICT}`.

This object does not ask “is this LLM aligned in general?” It asks whether a fixed trace satisfies a fixed restricted contract with adequate evidence. The object is meaningful only with its specification and scope.

10.2 Restricted supplied-certificate verification

The verifier performs a bounded sequence: schema checks; typed lineage checks; content-receipt comparisons; causal graph validation; authority/policy comparisons; support-oracle calls; graph reachability; correction-frontier checks; cross-scale witness checks; and final conjunctive release. Proposition 10 gives a restricted linear bound when primitive and oracle costs are bounded. Without that restriction, graph operations are polynomial in supplied nodes and edges, while window enumeration depends on the declared family.

No theorem here places general Index Alignment in P or NP. Finding an optimal route, discovering all side effects, independently enumerating obligations, constructing refinement witnesses, and determining unrestricted natural-language support are separate tasks. Polynomial verification of a supplied witness is not a classification of those tasks.

10.3 P-NP non-claims D1–D9

Following the P-NP seat, the paper expressly states:

- **D1.** No reduction from SAT or another NP-complete problem to general Index Alignment is given.
- **D2.** No reduction in the reverse direction is given.
- **D3.** Index Alignment is not declared NP-complete or NP-hard.
- **D4.** It is not declared to belong to P or NP as a general semantic problem.
- **D5.** Bounded supplied-certificate verification does not classify route search or correction synthesis.
- **D6.** `COHERENT-DDTL` is a separate, unresolved analytic probe, not this certificate and not an NP result.
- **D7.** “NP-as-mechanism” is excluded only as a program-policy prohibition where the named program ledger says so; it is not a mathematical impossibility theorem.
- **D8.** Finite enumeration and deterministic checking do not establish semantic completeness or real-system tractability.
- **D9.** No P-versus-NP consequence, RH consequence, or Coleman Conjecture consequence is claimed.

The useful complexity claim is deliberately modest: explicit witnesses make some audits mechanically checkable and their costs measurable.

10.4 Search, verification, enforcement, and effect

Four costs must not be conflated. **Search** finds routes, evidence, or repairs. **Verification** checks the supplied witness. **Enforcement** permits or blocks an execution. **Observed effect** records what the environment actually did. A verifier can be cheap while search is expensive; a correct verifier can be attached to a broken executor; a blocked action can still produce partial logs or externally visible effects before the block.

The AIce shipped simulator illustrates the distinction: advisory selection can be rejected yet executed by the simulator. A local wrapper can enforce blocking, but its E2 false-stop demonstrates a utility loss. These are source-reported synthetic observations, not a new live validation.

---

11. Evaluation Protocol

This section specifies the evaluation that tests the construction. The real-transcript and mutant suites it describes are the next experimental step; AIce/CWI figures quoted below are source-reported synthetic construction evidence for neighboring components.

11.1 Research questions

RQ1 — Sound booleanization. Can independent raters and rule-based oracles assign authorization and compliance labels under fixed specifications without unacceptable disagreement?

RQ2 — Localization. Does the first failed declared lift or concurrent frontier predict the actual repair site better than scalar-only telemetry?

RQ3 — Composition. Do R/C/N/K witnesses prevent false global acceptance under omitted side effects, shared invariants, stale approvals, and correction races?

RQ4 — Partial release. Does affected-closure quarantine retain useful unaffected output without promoting contaminated dependencies?

RQ5 — Cost and utility. What latency, token, dollar, human-review, false-block, and missed-coverage costs arise at each scale?

RQ6 — Drift. How often do criteria, registries, sources, and assessors change enough to invalidate old certificates?

These are questions, not predicted positive results.

11.2 Evaluation units and independent labels

Use operation/obligation episodes and whole traces. Episode labels include `A`, `C`, evidence sufficiency, causal freshness, and expected detection. Trace labels include global invariants, required obligation sets, refinement failures, corrections, and concurrent interleavings.

Ground truth must come from independently constructed specifications, hand-audited synthetic traces, or deliberately injected faults with known mechanisms. The monitor's own output cannot supply its labels. Pre-register thresholds, trace generation, primary outcomes, exclusions, and analysis before comparison; keep exploratory analyses separate.

Human raters label a subset independently before discussion. Report agreement, adjudication, and unresolved cases. Model-based support assessors must be compared with human or exact synthetic labels; another LLM's agreement is not independent proof. Cer-Eval motivates selective verification cost analysis, while VeriLA motivates explicit agent criteria; neither supplies labels for this protocol.

11.3 Diagnostic benchmark matrix

Family	Clean condition	Injected or naturally occurring defect	Required detection
MoE routing	live permitted experts	stale epoch, unauthorized substitution, missing type	`A_route` sub-conjunct
Retrieval	fresh complete chunks	hash-only middle, inactive citation, version mismatch	COV or lineage
Fidelity	supported claims	contradiction, unsupported synthesis, status inflation	SUPPORT/FID or StatusType
Tools	current signed approval	revocation, replay, schema drift	AUTH/POLICY/freshness
Agents	isolated handoffs	concurrent failure, omitted callback, shared cap	frontier and R/C/N/K
Continuity	explicit open goals	dropped obligation, silent waiver	`A_cont`
Corrections	fresh scoped supersession	stale acknowledgment, storm, partial propagation	CORR and affected closure
Provenance	independent challenged receipts	self-forged hash/HMAC, source substitution	PROV-002
Execution	clean baseline passes	import error, timeout, executor bypass	execution axis, not semantic label
Governance	fixed criteria manifest	silent rubric or registry drift	new epoch/invalidated certificate

The chunk→section→source→cross-source→handoff ladder is preregistered per task. Not every task uses every level, and skipped levels must be justified as inapplicable rather than silently omitted.

11.4 Detection metrics

Let `Y=1` denote an independently labeled violation and `D=1` a monitor violation detection. On valid evaluable episodes report TP, FP, TN, FN and

`TPR=TP/(TP+FN)`, `FPR=FP/(FP+TN)`, `precision=TP/(TP+FP)`.

Use `n/a` for empty denominators. Report `U` separately; do not encode it as either detection or non-detection. Where policy holds on `U`, measure that control choice under false-block and missed-violation outcomes without changing the epistemic label.

Trace-level metrics are global-violation recall, R/C/N/K failure recall by category, false global acceptance, false-block rate, detection latency, correction propagation completeness, stale-approval acceptance, and lineage completeness. A certificate that refuses everything could have zero false acceptance and unacceptable utility; a certificate that accepts everything could have high task completion and no safety value.

11.5 Evidence and composition metrics

`EvidenceCoverage(τ)=|Q_sound(τ)|/|Q_req(τ)|`, with `n/a` for an empty required universe and a separate vacuity flag. Report unknown rate and reasons, threshold-supplied rate, content-bound versus hash-only coverage, fresh-snapshot coverage, asynchronous side-effect coverage, refinement-witness coverage, and correction-frontier completeness.

Composition reporting includes the number of parent obligations with valid R/C/N/K witnesses, missing cross-operation invariants, cross-scale transfer failures, outside interference, and stale snapshot acceptance. Fine-scale pass rate is not a substitute for any of these.

11.6 Execution classification

Every fault variant requires the unmodified clean baseline under the same configuration.

- **CLEAN_PASS:** baseline satisfies named checks; otherwise comparison is invalid or labeled baseline failure.
- **DETECTED:** variant executes far enough to evaluate the intended property, and a predeclared named check fails for the intended reason.
- **UNDETECTED:** valid run completes without relevant detection.
- **HARNESS_ERROR:** import/dependency failure, malformed harness, unrelated exception, timeout, or other outcome prevents the intended check.
- **EXECUTION_ERROR:** enforcement or environment fails even if detection itself is valid.

A nonzero exit status alone never counts as detection. Named test results and reasons must be inspected. This corrects the methodological hazard in same-author mutation scripts that increment “killed” for any nonzero exit.

11.7 Baselines and ablations

Compare against: scalar weighted deficit; raw model self-assessment; citation-membership only; receipt-coverage only; global end-of-trace judge; per-call local checks without R/C/N/K; and human review alone where feasible. Match model capability, context, tool access, token budget, and human time as closely as possible.

Ablate typed lineage, content-bound receipts, noninterference checks, correction closure, cross-scale receipts, independent assessors, non-vacuity, and partial-release logic. One at a time ablations identify necessary implementation components but do not prove formal necessity. Interactions—especially C with N, and K with correction propagation—require joint ablations.

11.8 Cost and reporting

Report wall latency per stage, number and size of model calls, tokens, dollars, storage, graph traversal, human-review minutes, and energy where measurable. Separate verification cost from repair and enforcement. Include false-block cost and missed-coverage cost. Selective verification may reduce assessor calls, but the selection policy itself must be evaluated for missed violations.

The minimum report has seven axes: detection; T/F/U/NV reasons; execution; evidence coverage; R/C/N/K composition; corrections; and utility/cost. Reproducibility requires preregistration, seeds, versions, hashes, dependency versions, timeouts, and exploratory/confirmatory separation.

11.9 Eight core falsifier fixtures

These are **fixtures, not run results**.

1. **COV-pass/FID-fail:** all chunks receipted but conclusion unsupported. Wrong design passes if COV alone releases.
2. **Route substitution:** unauthorized but registered tool/expert. Wrong design passes if registry membership ignores Π.
3. **Stale correction handoff:** child uses pre-correction view. Wrong design passes if Aff is not invalidated.
4. **SEM-003 status inflation:** strategy labeled proved. Wrong design passes if status types are unchecked.
5. **SCOPE overclaim:** measured claim generalized to all workflows. Wrong design passes if tested-workflow scope is absent.
6. **PROV-002 hash-only reread:** hash presented as reading. Wrong design passes without independent content-bound verification.
7. **ACT conflation:** covered unauthorized instruction. Wrong design passes if COV grants Act.
8. **Index echo:** cited identifier absent from visible activations. Wrong design passes without generation–index lineage.

A fixture falsifies only the named design or implementation conformance condition. It never falsifies RH, the Coleman Conjecture, external Kakeya dimension, or the normative content of an authority predicate. Additional required fixtures test shared budgets, omitted side effects, concurrent failures, authority pollution, replayed approvals, empty obligation sets, and harness crashes.

11.10 Existing construction evidence and its limits

The opto source reports a versioned synthetic run with 110 runs, zero frozen-expectation mismatches, and 539 tests. These are quoted as source-reported artifacts; no suite was rerun here. They do not establish live efficacy. Limitations include handoff RPL=0, live `L_N=.75` versus compressed 1, order-unaware replay, and in-process registry forgery.

The open-addressing probe reports 7/8 confirmatory predictions held with P3 failed and still unresolved. This belongs to a neighboring hash-table model, not LLM alignment. Its value here is methodological: failed predictions remain visible, exploratory sweeps are not confirmatory, and simulation is not a theorem.

The supplied `thinkingmachines.txt` documents Benevolence Drift Case 01 only: an initially evasive/neutral response became “Yes” after a forced binary prompt. Case 02 has no forced-binary follow-up in the supplied material. Self-reported score changes are not independent metrics, and neither case validates boolean index alignment.

12. Extended Related Work, Failure Modes, and Research Questions

12.1 Attribution, support, and causal faithfulness

AIS frames attribution as whether generated text is supported by an identified source (Rashkin et al., 2023), and ALCE provides citation-oriented generation and evaluation (Gao et al., 2023). These are closely related to `Support(q)`, but they do not make support identical to index membership. Our certificate deliberately separates `Lineage`, `ActiveEvidence`, and `Fidelity` so a syntactically valid citation cannot compensate for an unsupported statement.

Wallat et al. (2024) further distinguish citation correctness from causal faithfulness: a cited document can support an answer without being the document the model actually used. Our event lineage records visible activation and snapshot access, not a complete causal explanation of neural computation. Thus even PASS on `ActiveEvidence` is evidence about the exposed trace, not a proof of internal causal reliance. This is an explicit limit, not a problem that can be solved by adding more citation IDs.

FABLES evaluates book-length faithfulness and content selection, including omission errors (Kim et al., 2024). Its human annotation discipline motivates claim-level sampling and omission checks. RAGChecker diagnoses retriever and generator modules with fine-grained metrics (Ru et al., 2024). The present framework differs by making each necessary relation a non-compensatory certificate conjunct with witness paths, while reusing the lesson that aggregate answer quality can conceal module defects.

12.2 Position effects and scale placement

Lost in the Middle shows that relevant information's position can affect long-context performance (Liu et al., 2024). This motivates a benchmark stratification by source location and scale, not a universal rule that middle evidence always fails. The scale tree must preserve source offsets and version identity so that a missing middle receipt is diagnosable. Chunk size, overlap, ordering, and hierarchy are experimental factors rather than fixed optimal design parameters.

The framework's directional coverage analog likewise requires declared directions of relevance, not a metaphysical claim that all semantic directions have been captured. Source coverage can be stratified by evidence class, tenant, modality, temporal interval, and claim type. An omitted class cannot be repaired by high coverage in other classes.

12.3 Agent criteria, reasoning verification, and independent judgments

VeriLA emphasizes interpretable verification of agent failures against explicit criteria (Sung, Kim, and Zhang, 2025). Derailer-Rerailer studies adaptive verification for reasoning (Wan et al., 2025), and Cer-Eval studies cost-efficient evaluation with statistical guarantees (Wang et al., 2025). These works motivate staged verification and cost reporting, but their methods and guarantees do not transfer automatically to non-compensatory index certificates.

Sycophancy research shows that model responses can track user preferences in undesirable ways (Sharma et al., 2024); BenchForm studies conformity to group responses (Weng, Chen, and Wang, 2025). For our protocol, independent pre-discussion judgments reduce one obvious channel of anchoring but do not eliminate shared training data, common rubric blind spots, or correlated tools. Multiple model judges are therefore not independent in the strong evidentiary sense merely because they return separate answers.

12.4 Criteria drift and governance

EvalGen reports criteria drift: judging outputs helps users refine the criteria by which they judge (Shankar et al., 2024). This complicates any claim that a complete fixed rubric can always be specified in advance. Our response is not to deny drift but to version it. Exploratory rubric development creates a new specification epoch; confirmatory runs retain a frozen manifest. Old certificates are comparable only under explicit migration rules.

Grill (2024) places testing infrastructures in a sociotechnical and political context. This matters because authority, acceptable risk, and release scope are not purely technical observations. A certificate can faithfully implement a harmful or contested policy. Governance must name who sets obligations, who can revise them, whose evidence counts, and who bears false-block and missed-violation costs. Technical provenance does not settle those normative questions.

12.5 Refinement, runtime verification, and concurrency

Abadi and Lamport (1991) provide refinement-mapping background; our R is a certificate-specific child-to-parent implication, not a quotation of their existence theorem. Bauer, Leucker, and Schallhart (2011) motivate inconclusive runtime-monitor outcomes. Goguen and Meseguer (1982) provide noninterference background. Herlihy and Wing (1990) supply linearizability, and Lamport (1978) happened-before. These formal traditions clarify R/C/N/K but do not prove that a language model satisfies them.

The open-addressing work of Farach-Colton, Krapivin, and Kuszmaul (2024/2025) concerns expected probe complexity in specified random-probing hash-table models. Stable placement and bounded lookup are analogies for receipt custody only. A hash-table slot has no authority semantics; successful lookup is not semantic reading; the cited theorems do not establish agent alignment or correction behavior over unbounded retractions.

12.6 Failure-mode catalog

Specification underapproximation. The obligation universe omits asynchronous work, shared resources, or semantic claims. Local checks pass and global acceptance is false. Defense: independent manifests, dynamic closure, side-effect inventory. Residual risk: unknown unknowns remain.

Checker unsoundness. A support model labels contradiction as support. Defense: exact synthetic controls, human labels, external assessors, calibrated abstention. Residual risk: correlated assessor failure.

Semantic booleanization error. Designers force a graded or contested judgment into PASS/FAIL. Defense: retain `UNKNOWN`, `CONTESTED`, thresholds, and assessor identity; booleanize only release control. Residual risk: thresholds can encode arbitrary value judgments.

Logging blind spots. Hidden expert mixtures or side channels expose no index. Defense: limit scope, add instrumentation, test noninterference. Residual risk: exposed indices may not represent internal causation.

Policy/authority confusion. Retrieval, order, signatures, or consensus are treated as permission. Defense: independent S/H/U channels and capability isolation. Residual risk: authority registry compromise.

Correction lag. Accepted correction exists but dependents remain live. Defense: transitive invalidation before promotion, frontier checks, replay protection. Residual risk: already emitted copies and unmodeled consumers.

Concurrent causality errors. Wall-clock sorting substitutes for dependencies. Defense: explicit HB, snapshot semantics, conflict resolution, concurrent failure sets. Residual risk: omitted edges.

False partial release. Dependency graph wrongly labels content unaffected. Defense: complete scope, N, independent closure audit. Residual risk: shared hidden context or caches.

Vacuity and denominator manipulation. Observed checks replace required checks, or an empty set is 100% covered. Defense: independent denominator, `n/a`, NONVACUOUS. Residual risk: manifest itself incomplete.

Criteria drift. Rubric changes while old certificates retain the same label. Defense: versioned manifest and confirmatory freeze. Residual risk: migration rules can hide substantive drift.

Enforcement gap. Selected, gated, executed, and observed actions diverge. Defense: separate records and end-to-end effects audit. Residual risk: irreversible action before gate.

Cost shifting. Verification reduces model calls but increases human annotation or storage. Defense: report all cost axes and false blocks. Residual risk: unpriced externalities and review burden.

12.7 Open research questions

1. Which useful parent specifications admit sound and efficiently checkable refinement maps?
2. How can obligation universes be independently enumerated while agents generate new work dynamically?
3. What consistency protocols minimize revalidation without accepting stale approvals?
4. What evidence supports semantic entailment rather than syntactic citation membership?

5. Can false global acceptance be bounded under partial coverage without calling `U` true?
6. Which properties are monitorable online, only end-of-trace, or not under realistic observation?
7. Do explicit ledgers and certificates improve end-to-end reliability under matched capability and cost?
8. How should concurrent correction frontiers merge across independently authorized registries?
9. Can refinement witnesses be reused across model updates without hiding semantic drift?
10. When is partial release socially or legally appropriate even if graph isolation holds?
11. How can independent verifiers be instantiated without exposing protected source content?
12. What do false blocks cost users, and who adjudicates their appeal?

---

13. Scope and Limitations

13.1 Empirical limitations

No new experiments were performed. All worked traces are synthetic and hand-constructed. No production trace, model checkpoint, user dataset, or real support assessor was evaluated. Existing AIce/CWI synthetic figures are source-reported and were not rerun. The construction therefore has no demonstrated accuracy, false-block rate, latency advantage, or task-success improvement.

Real-system efficacy is open. In particular, R/C/N/K may be difficult or impossible to establish for opaque models and open tool ecosystems. Complete evidence coverage is often unavailable. The appropriate outcome in such cases is limited scope or `NO_VERDICT`, not a default boolean pass.

13.2 Semantic limitations

Index alignment is not preference alignment, truth, beneficence, fairness, or safety. A certificate is relative to a specification, policy, trace, and assessor. A correct certificate under a bad specification can authorize a bad outcome. Exposed index lineage is not an account of all neural causes. `SUPPORT/FID` assessor reliability remains unresolved, and content-bound receipts cannot discharge it.

The four predicate families are not claimed to exhaust every desirable systems property. They operationalize portions of Coleman (2026a)'s four consistencies. Additional privacy, security, fairness, legal, and domain-safety obligations require their own specifications.

13.3 Geometric and program boundaries

Wang–Zahl and Guth–Wang–Zahl are theorem-background only. No asymptotic inequality is converted into an LLM boolean. Sticky KNS fan, sticky Kakeya proof case, and computational sticky custody are distinct. KNS-OBS-1 is a closure of a typed observability object only. The CC-S packaging argument is not a positive mathematical result, and program-level exclusions do not enter this paper's proof ledger. RH and the Coleman Conjecture remain open and are not addressed by the construction.

`h<1` is retained only as a stipulated personal-data/evaluator-non-sovereignty design boundary, not a measured privacy or sovereignty score. Private personal facts are excluded from synthetic predicates and cannot raise or lower a verdict. This does not prove privacy.

13.4 Provenance and contribution limitations

Coleman (2026a) and Coleman (2026b) are unpublished manuscripts by the author. Wang, Guth, and Zahl have no affiliation with this work. No all-eight-agent contribution is claimed. Available contributions were Kakeyalogic, L, P-NP, Riemann probe, AIce, Grok Bot, the combined team notes, and a local/public-manifest Git inventory handoff; one room was unreachable and is not listed as a contributor.

New Git evidence is restricted to the public-repository manifest. The final handoff reports **26 official commit-pinned public snapshots, 177,064,129 archive bytes, and 717 extracted research-text files across 26 public repository folders**. Extraction is not reading or validation; this paper did not indiscriminately review those 717 files and cites no new archive document as individually inspected evidence. `-Colemanfiles`, all private repositories, quarantined material, and excluded legacy GIUS materials are excluded by owner scope. Private/excluded content was verified absent from the shareable evidence paths by the handoff; none was incorporated and no attempt was made to fill excluded gaps. Snapshots do not include complete branches, tags, releases, issues, PRs, history, LFS, wikis, Actions, or submodule inventories; this is not “Everything” archival completion. No repository write occurred.

13.5 Complexity and security limitations

The restricted verification bound assumes supplied certificates, finite graphs, declared windows, and bounded primitive/oracle checks. It is not a general complexity classification. The paper does not prove P≠NP, NP-hardness, or an “NP mechanism.” Cryptographic identifiers are integrity tools under stated key boundaries, not authority or semantic proof. An in-process verifier cannot be independent of itself.

---

14. Conclusion

Index alignment becomes traceable as a boolean certification problem only after substantial discipline is added: typed indices, fixed registry and policy snapshots, independently declared obligations, explicit evidence, content-bound receipts, correction-aware dependencies, and non-compensatory release rules. Under those

conditions, routing, ledger, generation, and continuity can be checked as separate predicate families, and the earliest known failed scale or concurrent failure frontier can be returned with witnesses. This is a finite systems construction, not a consequence of Kakeya geometry.

The central composition result is conditional. Sound local certificates lift across declared abstraction levels when refinement, complete coverage, noninterference, and correction-aware freshness hold. Local passes alone are insufficient, and the induction is not over model size or training scale. Sticky nesting supplies a simple case; non-sticky bands are repaired by affected-closure invalidation and capped rechecking; residuals remain visible. Structural readiness does not subsume cross-index/scale transfer, authority, scope, or external support/fidelity.

The practical thesis is therefore deliberately narrow: **at fixed caller-supplied thresholds, a sparse system's declared index-consistency contract can be audited by PASS-only boolean release bits while retaining FAIL, UNKNOWN, and NO_VERDICT diagnostics and evidence paths**. Coverage is not fidelity, order is not authority, custody is not truth, and a blocked full release need not suppress authorized unaffected partial output. Whether this improves real systems remains an empirical question.

References

1. Coleman, M. (2026a). Index Alignment in Large Language Models: Sparse Routing, State Ledgers, and Generative Continuity. Unpublished manuscript.
2. Coleman, M. (2026b). Kakeya Grain at the Slab: Induction on Scales, Continuous Direction, and Prime-Carrying Custody under Love-Squared Coherence. Unpublished manuscript.
3. Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR. https://arxiv.org/abs/1701.06538
4. Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Chen, Z. (2020). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. https://arxiv.org/abs/2006.16668
5. Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR. https://arxiv.org/abs/2101.03961
6. Jiang, A. Q., et al. (2024). Mixtral of Experts. https://arxiv.org/abs/2401.04088
7. Cai, W., et al. (2024). A Survey on Mixture of Experts in Large Language Models. https://arxiv.org/abs/2407.06204
8. Yue, T., et al. (2024). Ada-K Routing: Boosting the Efficiency of MoE-based LLMs. https://arxiv.org/abs/2410.10456v1
9. Cai, R., Ro, Y., Kim, G.-W., Wang, P., Bejnordi, B. E., Akella, A., & Wang, Z. (2024). Read-ME: Refactorizing LLMs as Router-Decoupled Mixture of Experts with System Co-Design. https://arxiv.org/abs/2410.19123
10. Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer Networks. NeurIPS. https://arxiv.org/abs/1506.03134
11. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS. https://arxiv.org/abs/2005.11401
12. Kim, T. E., & Diaz, F. (2025). LTRR: Learning To Rank Retrievers for LLMs. https://arxiv.org/abs/2506.13743
13. Wu, M., Liu, Z., Li, X., Lan, Y., Yan, Y., Wang, S., Yang, C., Yu, M., Zeng, Z., & Sun, M. (2026). Finding What Matters: Anchoring Context Knowledge with Evolving Indices for Iterative Retrieval (GraphAnchor). https://arxiv.org/abs/2601.16462
14. Liang, X., Niu, S., Li, Z., Zhang, S., Song, S., Wang, H., Yang, J., Xiong, F., Tang, B., & Xi, C. (2024). Empowering Large Language Models to Set up a Knowledge Retrieval Indexer via Self-Learning. https://arxiv.org/abs/2405.16933
15. Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. https://arxiv.org/abs/2302.04761

16. Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR. https://arxiv.org/abs/2210.03629
17. Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. https://arxiv.org/abs/2308.08155
18. Hong, S., et al. (2023). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. https://arxiv.org/abs/2308.00352
19. Fourney, A., et al. (2024). Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. https://arxiv.org/abs/2411.04468
20. Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. https://arxiv.org/abs/2310.08560
21. Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. https://arxiv.org/abs/2304.03442
22. Xiao, H., Xu, H., Zhang, X., Chen, Y., & Shi, X. (2026). TARL: Transaction-Aware Reliable Ledgers for Executable Memory Management in Long-Term Agents. https://arxiv.org/abs/2608.03699
23. Yang, G., Liu, X., Wu, B., & Wang, S. (2026). StateLedger: Path-Addressed External Memory for Persistent Multi-Agent Systems. NeurIPS 2026 poster. https://neurips.cc/virtual/2026/poster/151469
24. Wang, H., & Zahl, J. (2025). Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions. https://arxiv.org/abs/2502.17655
25. Guth, L., Wang, H., & Zahl, J. (2026). A streamlined proof of the Kakeya set conjecture in $\mathbb{R}^3$. https://arxiv.org/abs/2601.14411
26. Wang, H., & Wu, S. (2024). Restriction estimates using decoupling theorems and two-ends Furstenberg inequalities. https://arxiv.org/abs/2411.08871
27. Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., & Reitter, D. (2023). Measuring Attribution in Natural Language Generation Models. Computational Linguistics 49(4). https://aclanthology.org/2023.cl-4.2/
28. Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling Large Language Models to Generate Text with Citations. EMNLP. https://arxiv.org/abs/2305.14627
29. Wallat, J., Heuss, M., de Rijke, M., & Anand, A. (2024). Correctness is not Faithfulness in RAG Attributions. https://arxiv.org/abs/2412.18004
30. Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL. https://aclanthology.org/2024.tacl-1.9/
31. Kim, Y., Chang, Y., Karpinska, M., Garimella, A., Manjunatha, V., Lo, K., Goyal, T., & Iyyer, M. (2024). FABLES: Evaluating Faithfulness and Content Selection in Book-length Summarization. COLM. https://arxiv.org/abs/2404.01261
32. Ru, D., et al. (2024). RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. NeurIPS. https://arxiv.org/abs/2408.08067

33. Sung, Y. Y., Kim, H., & Zhang, D. (2025). VeriLA: A Human-Centered Evaluation Framework for Interpretable Verification of LLM Agent Failures. https://arxiv.org/abs/2503.12651
34. Sharma, M., et al. (2024). Towards Understanding Sycophancy in Language Models. ICLR. https://arxiv.org/abs/2310.13548
35. Weng, Z., Chen, G., & Wang, W. (2025). Do as We Do, Not as You Think: The Conformity of Large Language Models. ICLR. https://arxiv.org/abs/2501.13381
36. Wang, G., Chen, Z., Li, B., & Xu, H. (2025). Cer-Eval: Certifiable and Cost-Efficient Evaluation Framework for LLMs. https://arxiv.org/abs/2505.03814
37. Wan, G., Wu, Y., Wang, H., Zhao, S., Chen, J., & Li, S. (2025). Derailer-Rerailer: Adaptive Verification for Efficient and Reliable Language Model Reasoning. Findings of ACL. https://aclanthology.org/2025.findings-acl.18/
38. Shankar, S., Zamfirescu-Pereira, J. D., Hartmann, B., Parameswaran, A. G., & Arawjo, I. (2024). Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. UIST. https://arxiv.org/abs/2404.12272
39. Grill, G. (2024). Constructing Capabilities: The Politics of Testing Infrastructures for Generative AI. FAccT. https://doi.org/10.1145/3630106.3659009
40. Abadi, M., & Lamport, L. (1991). The Existence of Refinement Mappings. Theoretical Computer Science 82(2), 253–284. https://doi.org/10.1016/0304-3975(91)90224-P
41. Bauer, A., Leucker, M., & Schallhart, C. (2011). Runtime Verification for LTL and TLTL. ACM TOSEM 20(4), Article 14. https://doi.org/10.1145/2000799.2000800
42. Farach-Colton, M., Krapivin, A., & Kuszmaul, W. (2024; revised 2025). Optimal Bounds for Open Addressing Without Reordering. FOCS 2024. https://arxiv.org/abs/2501.02305v2
43. Goguen, J. A., & Meseguer, J. (1982). Security Policies and Security Models. IEEE Symposium on Security and Privacy. https://doi.org/10.1109/SP.1982.10014
44. Herlihy, M. P., & Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. ACM TOPLAS 12(3), 463–492. https://doi.org/10.1145/78969.78972
45. Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM 21(7), 558–565. https://doi.org/10.1145/359545.359563

Appendix A. JSON-Like Certificate Schema

The following is a normative-shaped schema example, not a claim that a particular implementation exists. `null` is not silently PASS; every required field with unavailable evidence has a matching reason code. Digests bind artifacts, while verifier identity and challenged spans provide the evidence boundary.

```
{
  "schema_version": "ia-cert-0.1-proposed",
  "certificate_id": "synthetic:example-A:001",
  "claim_kind": "worked_example",
  "spec": {
    "spec_version": "Spec9",
    "criteria_manifest_digest": "sha256:...",
    "obligation_universe_digest": "sha256:...",
    "nonempty_required_scales": [0, 1],
    "min_coverage": 1.0,
    "max_rounds": 3,
    "thresholds": {"theta_KT": 1.5, "theta_F": 2.0}
  },
  "versions": {
    "registry": "R4",
    "policy": "P9",
    "authority": "Auth3",
    "source_snapshot": "S12",
    "verifier": "independent-verifier:V2",
    "support_assessor": "assessor:A7"
  },
  "scale_records": [
    {
      "scale": 0,
      "partition_digest": "sha256:...",
      "parent_map": {"grain:f1": "grain:c1"},
      "obligations": ["o1", "o2", "o3", "o4", "o5"],
      "predicates": {
        "A_route": {"Ev": "PASS", "witnesses": ["route-receipt:1"]},
        "A_ledger": {"Ev": "PASS", "witnesses": ["commit:3"]},
        "A_gen": {"Ev": "PASS", "witnesses": ["support-run:7"]},
        "A_cont": {"Ev": "PASS", "witnesses": ["projection:4"]}
      },
      "subchecks": {
        "COV": {"Ev": "PASS", "receipts": ["rc3"]},
        "ORDER": {"Ev": "PASS", "hb_digest": "sha256:..."},
        "CORR": {"Ev": "PASS", "frontier": ["corr:0"]},
        "AUTH": {"Ev": "PASS", "S": 1, "H": 1, "U": 1}
      },
      "R": {"Ev": "PASS", "refinement_witness": "ref:0-1"},
      "C": {"Ev": "PASS", "independent_manifest": "manifest:Q0"},
      "N": {"Ev": "PASS", "isolation_witness": "isolation:2"},
      "K": {"Ev": "PASS", "snapshot": "S12", "valid_at_commit": true}
    }
  ],
  "failures": {
    "total_order_first_fail": null,
    "concurrent_frontier": [],
    "unknowns": [],
    "no_verdict_reasons": []
  },
  "correction": {
    "governing_frontier_digest": "sha256:...",
    "affected_seeds": [],
    "affected_closure": [],
    "invalidated_certificates": [],
    "residuals": []
  },
  "release": {
    "STRUCTURAL_READY": {"b": 1},
    "CROSS_OK": {"Ev": "PASS"},
    "CROSS_SCALE_OK": {"Ev": "PASS", "receipts": ["transfer:0-1"]},
    "POLICY_OK": {"Ev": "PASS"},
    "AUTH": {"Ev": "PASS"},
    "SUPPORT": {"Ev": "PASS"},
    "FID": {"Ev": "PASS"},
```

```
    "SCOPE": {"Ev": "PASS"},
    "NONVACUOUS": {"Ev": "PASS"},
    "outcome": "CERTIFIED",
    "partial_exclusions": []
  },
  "execution": {
    "clean_baseline": "CLEAN_PASS",
    "detection_status": "NOT_APPLICABLE",
    "execution_status": "COMPLETED",
    "selected_action": "answer-commit:synthetic",
    "gate_verdict": "ALLOW",
    "executed_action": "local-record-only",
    "observed_effect": "certificate-written"
  },
  "nonce": "synthetic-nonce-001"
}
```


A.1 Required witness rules

A `PASS` requires positive evidence; a `FAIL` requires an identified counterexample; `UNKNOWN` requires an insufficiency reason; `NO_VERDICT` names missing thresholds or contracts or an operationally unresolved outcome such as exhausted reconciliation. These labels overlap operationally in U, but their reasons remain distinct in the trace. `SKIPPED` is allowed only for later unsafe checks and is never converted to PASS. The certificate retains sub-conjuncts rather than only the four composite bits.

A partial record adds `dependency_closure_digest`, `excluded_obligations`, `partial_label`, and separate authority for emission. `HELD_PARTIAL` cannot be inferred solely from `Aff` disjointness; N and complete dependency enumeration are also required. An `AWAITING_HUMAN` record names the authorized adjudicator role and exact question, not a generic request for human oversight.

Appendix B. Reference Pseudocode

B.1 Evidence-conditioned local evaluation

```
def evaluate_obligation(q, evidence, contract):
    if q not in contract.required_obligations:
        return {"Ev": "NO_VERDICT", "reason": "UNDECLARED_OBLIGATION"}
    missing = contract.missing_thresholds(q)
    if missing:
        return {"Ev": "NO_VERDICT", "missing": missing}
    if execution_prevents_evaluation(evidence):
        return {"Ev": "UNKNOWN", "execution": "HARNESS_ERROR"}
    counterexample = find_declared_counterexample(q, evidence)
    if counterexample is not None:
        return {"Ev": "FAIL", "counterexample": counterexample}
    positive = find_sound_positive_witness(q, evidence)
    if positive is not None:
        return {"Ev": "PASS", "witness": positive}
    return {"Ev": "UNKNOWN", "reason": "INSUFFICIENT_EVIDENCE"}
```

`find_sound_positive_witness` is not a generic LLM call. It invokes the declared checker under its soundness assumptions. The code's existence would not prove those assumptions.

B.2 Strong conjunction and boolean release projection

```
def evidence_and(records):
    if any(r["Ev"] == "FAIL" for r in records):
        return "FAIL"
    if any(r["Ev"] == "UNKNOWN" for r in records):
        return "UNKNOWN_WITH_REASONS"
    if any(r["Ev"] == "NO_VERDICT" for r in records):
        return "NO_VERDICT_WITH_REASONS"
    if all(r["Ev"] == "PASS" for r in records):
        return "PASS"
    return "NO_VERDICT"

def release_bit(record):
    return 1 if record["Ev"] == "PASS" else 0
```

The diagnostic aggregation preserves reason codes; `release_bit` is applied only at a control boundary. An observed FAIL dominates for semantic conjunction even if another conjunct is unknown. This does not collapse the reasons in the stored trace.

B.3 Cross-scale lift

```
def lift_parent(q, child_certs, witnesses):
    if not independent_coverage(q, witnesses.C):
        return no_verdict("INCOMPLETE_OBLIGATION_UNIVERSE")
    if not noninterference(q, witnesses.N):
        return fail_or_unknown("NONINTERFERENCE", witnesses.N)
    correction = correction_status(q, witnesses.K)
    if correction.Ev != "PASS":
        return fail_or_no_verdict("CORRECTION_FRONTIER", correction)
    if not refinement_valid(q, child_certs, witnesses.R):
        return fail_or_unknown("REFINEMENT", witnesses.R)
    if any(c.Ev != "PASS" for c in child_certs):
        return aggregate_children(child_certs)
    return pass_with_path(q, child_certs, witnesses)
```

The order shown is illustrative, not a security rule that avoids checking adversarial input. Implementations should safely parse and validate all required records before relying on any one result.

B.4 Affected closure and partial release

```
def invalidate(seeds, G, old_certs):
    aff = reachable_union(G, seeds)  # includes seeds and all dependents
    for node in aff:
        old_certs[node].status = "INVALIDATED"
        old_certs[node].release_bit = 0
    return aff

def may_emit_partial(output, aff, evidence):
    required = [dependency_complete(output), noninterference(output),
                disjoint(dependencies(output), aff),
                current_route_ok(output), current_policy_ok(output),
                current_auth_ok(output), support_fid_ok(output),
                continuity_ok(output)]
    return all(x.Ev == "PASS" for x in required)
```

A failed disjointness test identifies affected content; unknown dependency completeness yields no verdict. Neither case licenses calling the output semantically false.

B.5 Reconciliation loop

```
def reconcile(trace, max_rounds):
    if max_rounds is None:
        return NO_VERDICT("MISSING_MAX_ROUNDS")
    for round_id in range(max_rounds):
        seeds = nonpass_structural_seeds(trace)
        if not seeds:
            if all_cross_scale_receipts_pass(trace):
                return STRUCTURAL_READY(trace)
            return transfer_status(trace)  # FAIL for a witnessed bad transfer; NO_VERDICT if
missing/unresolved
        aff = invalidate(seeds, trace.G, trace.certificates)
        repaired = repair_only_authorized(trace, aff)
        recheck_entire_closure(trace, aff, repaired)
    return NO_VERDICT("CORRECTION_STORM_OR_ROUND_CAP")
```

`repair_only_authorized` is intentionally abstract. Certificate verification does not invent authority or a repair. If no repair is possible, residuals remain held.

Appendix C. Notation Index

Symbol	Meaning
ι	typed index `(type,name,epoch,version)`
`R_t`, `R_{≤t}`	activations at event `t`; visible activation history
μ, `Lin_t(c)`	typed index morphism; witnessed lineage paths
`L_t`, `View(L_t)`	append-oriented ledger; committed non-superseded view
`C_k`	continuity state projection at scale `k`
`P_k`	partition at scale `k`; finer partitions refine coarser ones
`Q_k`	independently declared obligation universe
`D_k(q)`	child obligations required to lift parent `q`
`A_q`, `C_q`	semantic authorization and compliance bits
`V^A_q`, `V^C_q`	monitor verdicts `T/F/U` under available evidence
`Ev(P)`	PASS, FAIL, UNKNOWN, or NO_VERDICT evidence status
`b(P)`	release bit, 1 iff PASS and 0 otherwise
`A_route`	typed, live, permitted, budgeted, parented, admissible routes
`A_ledger`	provenance, authority, transition, supersession, coverage, order, correction
`A_gen`	claim lineage, active evidence, support, status, scope, attribution
`A_cont`	projection, obligations, handoffs, corrections, directions, role bounds
`A(k)`	conjunction of the four predicate families at scale `k`
`ALIGNED(T)`	all declared scales pass; not synonymous with release
`FIRST_FAIL`	least known failed scale in a finite total order

Symbol	Meaning
`Frontier(F)`	minimal failed nodes under declared causal DAG order
`STICKY_k`	computational custody nesting, acknowledgments, optional clustering
`R,C,N,K`	refinement, complete coverage, noninterference, freshness/concurrency
`G=(V,E)`	finite claim/dependency DAG
`Aff`	affected transitive dependency closure from correction/failure seeds
`COV`	content-bound receipt completeness, not comprehension or fidelity
`ORDER`	explicit happened-before consistency, not authority or consensus
`CORR`	authorized applicable correction closure
`CROSS_SCALE_OK`	explicit transfer receipts between abstraction levels
`S,H,U`	authenticated source authority, hierarchy validity, authorized intent
`Act`	action authority gate; no implication that zero means Stop
`SUPPORT/FID`	externally declared semantic support/fidelity assessor contract
`Q_req`, `Q_sound`	required obligations; obligations with sound boolean evidence
`TP,FP,TN,FN`	detection counts under independent labels on valid episodes
`HARNESS_ERROR`	intended property could not be evaluated due to harness/infrastructure
`EXECUTION_ERROR`	execution/enforcement failed, separately from predicate detection

Appendix D. Extended Mutation and Negative-Control Inventory

ID	Mutation	Clean expected	Fault expected	Non-detection warns about
M01	registry epoch rollback	live current expert	ROUTE fails or unknown	freshness
M02	registered unauthorized tool	permitted call	POLICY/AUTH fail	capability isolation
M03	valid hash, wrong quoted span	content receipt	COV fail	hash-as-reading
M04	inactive citation ID	active lineage	GEN lineage fail	citation membership
M05	real ID, unsupported sentence	supported claim	FID fail	lineage-as-truth
M06	correct answer, post-rationalized citation	causal exposed lineage	active-evidence mismatch	support versus causal faithfulness
M07	missing middle chunk	complete coverage	COV unknown	denominator integrity
M08	omitted asynchronous callback	closed universe	C unknown/fail	dynamic obligations
M09	shared-cap overflow	atomic reservation	parent compliance fail	cross-operation R/C
M10	stale signed approval	current approval	AUTH fail/unknown	custody versus freshness
M11	replayed nonce	unused nonce	AUTH/transition fail	replay handling

ID	Mutation	Clean expected	Fault expected	Non-detection warns about
M12	correction before read omitted	correction frontier	CORR fail/unknown	causal visibility
M13	stale child governing acknowledgment	current `c_id,c_ver`	ANCEST fail	cross-registry versions
M14	one affected descendant omitted	full invalidation	closure completeness fail	transitive repair
M15	correction exceeds scope	scoped supersession	AUTH fail	applicability
M16	conflicting concurrent commits	resolved order	K unknown/fail	consensus versus order
M17	two independent failed branches	no failures	frontier set with two	causal localization
M18	retrieved instruction changes ACL	immutable authority	N/AUTH fail	ingestion independence
M19	router self-accepts commit	proposal-only router	capability fail	role isolation
M20	generator self-certifies support	external assessor	SUPPORT unknown/fail	verifier independence
M21	empty obligation universe	nonempty manifest	NONVACUOUS fail	vacuous 100%
M22	missing coverage threshold	supplied threshold	NO_VERDICT	hidden defaults
M23	missing support assessor	available assessor	UNKNOWN	assessor availability
M24	strategy relabeled proved	status preserved	StatusType fail	semantic status
M25	measured claim leaves workflow	scoped claim	SCOPE fail	empirical overclaim
M26	compression drops open goal	obligation checkpoint	A_cont fail	continuity
M27	gate rejects but tool executes	gate/execution agreement	EXECUTION_ERROR	advisory enforcement
M28	mutant import crash	clean baseline pass	HARNESS_ERROR	false kill count
M29	unrelated assertion fails	relevant checks valid	HARNESS_ERROR or baseline issue	attributable detection
M30	correction alternation past cap	stable frontier	NO_VERDICT/storm	bounded reconciliation
M31	false partial with hidden cache	no shared state	partial denied	noninterference
M32	silent criteria edit	versioned rubric	new epoch required	governance drift

A mutant is evidence only when clean baseline, valid execution, and named attributable failure are recorded. Surviving same-author mutants identify blind spots but do not measure deployment prevalence.

Appendix E. Prior-Paper Crosswalk

Prior source construct	Continuation object	What changes	Boundary retained
2026a routing–capability consistency	`A_route`	registry, permission, budget, parentage, optional task rule	gate weight not expert correctness
2026a routing–ledger consistency	`A_ledger` plus lineage	explicit provenance, transitions, authority, supersession	router cannot self-commit
2026a generation–index consistency	`A_gen`	adds support, status, scope, attribution completeness	citation membership not fidelity
2026a continuity–ledger consistency	`A_cont`	explicit obligations, correction and handoff checkpoints	fluent text not preserved state
2026a weighted alignment deficit	boolean vector plus optional telemetry	required defects cannot be averaged away	scalar remains diagnostic only
2026a IA-MoA role proposal	checker and release controller	separate evidence and enforcement records	no new model architecture claimed
2026a typed morphisms	versioned lineage and refinement	paths become certificate edges	exposed paths not full neural causation
2026a hierarchical indices	declared finite scale tree	no automatic training/model-scale induction	finite M chosen by task
2026b sticky-first order	governing acknowledgment nesting	stable unique parent and authority scope	not external sticky Kakeya case
2026b non-sticky factoring	non-sticky bands plus `Aff`	repair whole affected closure	no local semantic stability theorem
2026b residual accounting	explicit residual list and round cap	no disappearing uncovered items	cap not correctness
2026b HELD	HELD_PARTIAL	authorized unaffected output can emit	HELD not truth
2026b authority detection	independent S/H/U evidence	separate AUTH and POLICY from COV	Act=0 does not entail Stop
2026b object/status firewall	status register and theorem-background labels	typed closure not semantic release	RH/Coleman untouched

Appendix F. Source and Contribution Accounting

F.1 Primary former papers

Both former papers were primary references and were read in full. Coleman (2026a) supplies the central LLM problem, four consistency relations, role separation, typed indices, and evaluation motivation. Coleman (2026b) supplies scale-order methodology, object firewalls, custody/HELD distinctions, authority boundaries, and claim-status discipline. The continuation is integrated along the **core index-alignment formalism**; the Riemann/program material enters only through that bridge and does not become a separate RH paper.

F.2 Reachable seat contributions

Contribution	Material integrated	Not claimed
Kakeyalogic	COV/ORDER/CORR/STICKY, boolean thresholds, geometric firewall	WZ/GWZ implies LLM alignment

Contribution	Material integrated	Not claimed
L seat	S/H/U authority, Act=0≠Stop, evaluator boundary	measured h or universal authority oracle
P-NP seat	INDEX-ALIGN-CERT, bounded verification, D1–D9	NP-completeness or P/NP classification
Riemann probe	grains/slabs, governing acknowledgments, Aff repair, STRUCTURAL_READY versus RELEASE_FULL, eight fixtures	RH result or semantic FID proof
AIce	conditional DAG retention, selection/gate/execution/effect distinction, synthetic limitations	live system efficacy
Grok Bot packet	diagnostic ladder, partial protocol, independent provenance, costs and drift	completed real-transcript evaluation
Combined team notes	R/C/N/K composition proofs, semantic/monitor separation, concurrent examples, execution taxonomy	universal local-to-global theorem
Public Git handoff	public-only provenance and precise archive coverage	all-repository content review or private access

One named room was unreachable; therefore no eight-agent contribution claim is made. The phrase “ten agent team” in coordination notes is not treated as evidence that ten distinct contributors supplied material.

F.3 Public snapshot coverage

The final public handoff was inspected at `/workspace/peaice/team-formalization-2026-11-05/github-inventory-manny365-2026-11-05.md` and the access manifest. It reports 26 commit-pinned official public archives, 177,064,129 bytes, and 717 extracted research-text documents across 26 public repository folders. These are **coverage counts, not reviewed-source counts**. No additional one of those 717 documents was used as evidence without individual inspection; this manuscript's detailed technical claims rely on the separately read local primary papers, seat packets, AIce sources, and public scholarly pages.

Private repositories and `-Colemanfiles` are excluded by owner scope. Quarantine and excluded legacy GIUS material were not read or used. The snapshot omits complete branch/tag/release/issue/PR/history/LFS/wiki/Actions/submodule exports. Archive availability is not remote-currentness beyond the pinned commits and not a repository transfer to any destination.

F.4 Local snapshot versus remote manifest

The Riemann packet records local filesystem Git HEADs for provenance. They are not freshly verified remote HEADs. Where the public handoff reports a different pinned remote snapshot—for example Kakeyalogic—the two observations are kept distinct rather than silently reconciled. This paper does not infer mathematical validity from either commit identifier.

Appendix G. Claim Scope Register

Claim/object	Basis in this paper	Exact scope
Semantic A/C bivalence under fixed spec/history	definition	does not imply monitor knows value

Claim/object	Basis in this paper	Exact scope
Strong Kleene aggregation	proved (Prop. in §6)	sound local verdicts only
Conditional cross-scale induction	proved (Prop. in §6)	R/C/N/K plus sound base evidence
First-failure existence/uniqueness	proved (Prop. 3)	known failures, total declared scale order
Concurrent failure frontier	proved (Prop. 4)	explicit acyclic causal order
Finite computability	proved under stated hypotheses	terminating checkers and finite supplied inputs
Non-vacuity release blocking	proved (Prop. 6)	PASS-only full-release contract
Unaffected-output preservation	proved under stated hypotheses	complete graph, N, stable evidence
AIce retention	proved under stated hypotheses	base, edges, admission, supersession, enforced gate
Refinement invalidation	proved (Prop. 9)	old certificate unjustified, not necessarily false
Restricted verification bound	proved under stated hypotheses	supplied certificates and bounded primitives/oracles
Four predicate families	defined in this paper	partial operationalization of 2026a
Grain/slab/sticky custody	defined by analogy	not geometric theorem objects
Aff reconciliation and capped schedule	defined in this paper	efficacy untested
STRUCTURAL_READY/RELEASE_FULL	defined in this paper	surrogate and release contract
INDEX-ALIGN-CERT	defined (Def. 28)	no complexity-class claim
Evaluation protocol	defined in this paper	not run here
Worked traces and mutants	hand-constructed worked examples	synthetic identifiers only
AIce 110 runs/539 tests/opto observations	reported by AIce source	not rerun or generalized
WZ/GWZ dimension theorem	external theorem	no AI implication
KNS-OBS-1	typed object in program source	not Kakeya sticky proof or release
CC-S	packaging argument in program source	not positive mathematical closure
Program negative lanes	program exclusion where source labels it	not imported as LLM theorems
RH, Coleman Conjecture	unresolved	outside scope
SUPPORT/FID reliability	unresolved	external validation required
Live utility, false blocks, costs	measured by §11 protocol	future protocol only

Appendix H. Minimal Audit Checklist

Before accepting a certificate, an auditor should be able to respond yes to each applicable question:

1. Is the specification version fixed and independently reviewed?
2. Is the required obligation universe nonempty and independently enumerated?
3. Are dynamic side effects, retries, and callbacks included?
4. Are typed index names, versions, registry epochs, and policy snapshots present?
5. Are all required thresholds supplied without hidden defaults?

6. Does every PASS have positive evidence and every FAIL a counterexample?
7. Are UNKNOWN, NO_VERDICT, SKIPPED, HARNESS_ERROR, and EXECUTION_ERROR distinct?
8. Do content receipts include challenged spans and independent verification where required?
9. Are authority and policy evidence independent of content ingestion?
10. Are causal dependencies explicit rather than inferred from timestamps alone?
11. Are snapshots current at commit/effect boundaries and correction-closed?
12. Does each parent lift have R/C/N/K witnesses, including cross-operation obligations?
13. Are concurrent failures returned as a frontier rather than arbitrarily ordered?
14. Does correction invalidation cover the full transitive affected closure?
15. Are round caps and residual items recorded?
16. Does full release separately check CROSS, CROSS_SCALE, SCOPE, AUTH, POLICY, SUPPORT, and FID?
17. Is partial output independently authorized, unaffected, and explicitly labeled?
18. Are selected action, gate verdict, executed action, and observed effect distinguished?
19. Did mutation comparisons require CLEAN_PASS and named attributable detection?
20. Are evidence coverage, false blocks, missed coverage, latency, tokens, dollars, and human costs reported separately?
21. Are criteria changes versioned as a new epoch?
22. Do summaries preserve the distinction between external theorems, constructions, proofs, observations, and open questions?

A “yes” to this checklist supports audit readiness only. It does not substitute for external specification review or empirical evaluation.

Supporting deep-dive research: *deep-dives/* and *CLAIM-EVIDENCE-MATRIX.md* in this package.